



Movent API

Documentacion Tecnica del Backend

Go + Gin | UNIS 2026

Generado el 8 de abril de 2026



Indice de contenidos

GENERAL

- Inicio

API

- Controllers
- Services
- Repositories

ESTRUCTURA

- Models
- DTO
- Helpers

INFRAESTRUCTURA

- Middlewares
- Config
- Database

GENERAL

Inicio

Backend desarrollado en Go/Gin para la agencia de viajes Movent.
Proyecto final UNIS 2026.

Descripcion

API REST que integra multiples proveedores de vuelos y hoteles.
Desarrollada con Go, Gin y MySQL.

Tecnologias

- Go / Gin
- MySQL
- JWT para autentificacion
- Arquitectura por capas: controllers, services, repositories

Paquetes

- Controllers - Manejadores HTTP de cada endpoint
- Services - Logica de negocio
- Repositories - Acceso a base de datos
- DTO - Data Transfer Objects
- Models - Estructuras de datos
- Middlewares - Autenticacion y roles
- Helpers - Email, PDF, tokens, hash
- Config - Configuracion del servidor
- Database - Conexion a MySQL
- Server - Punto de entrada principal

API

Controllers

Controladores HTTP de la API de Movent. Cada controlador agrupa los handlers relacionados a un recurso o dominio especifico de la aplicacion.

Controladores HTTP de la agencia de viajes. Cada controlador recibe solicitudes de Gin, delega la logica de negocio al servicio correspondiente y devuelve la respuesta JSON al cliente.

FUNCTIONS

`func EnviarContacto(c *gin.Context)`

EnviarContacto

Procesa el formulario de contacto enviado desde el sitio web. Valida que el mensaje tenga al menos 10 caracteres, construye un correo HTML con los datos del remitente y lo envia a la bandeja de entrada de soporte de MOVENT.

Parametros:

- c: contexto de Gin con la solicitud HTTP

Retorna:

- HTTP 200 OK: JSON con mensaje de confirmacion al visitante
- HTTP 400 Bad Request: si faltan campos requeridos o el mensaje es muy corto
- HTTP 500 Internal Server Error: si ocurre un error al enviar el correo

Notas:

- Si el asunto llega vacío se usa el valor por defecto "Consulta desde el sitio web"
- El correo se envia a la cuenta configurada en el SMTP (cfg.User)

TYPES

`type AdminController struct {`

// Has unexported fields.

`}`

AdminController

Controlador que maneja los endpoints del panel de administracion, incluyendo gestion de usuarios, roles, proveedores, reservaciones recientes y metricas financieras.

```
func NewAdminController(db *sql.DB) *AdminController
    NewAdminController
```

Constructor que retorna una nueva instancia de AdminController con la conexion a la base de datos inyectada.

- db: puntero a la conexion de base de datos SQL
- *AdminController: puntero a la nueva instancia

```
func (ctrl *AdminController) ActualizarRol(c *gin.Context)
    ActualizarRol
```

Actualiza el rol de un usuario especifico. El ID del usuario se lee desde el parametro de URL :id y el nuevo rol desde el body JSON.

- HTTP 200 OK: JSON con mensaje de confirmacion
- HTTP 400 Bad Request: si el ID es invalido o el campo rolId no esta presente
- HTTP 500 Internal Server Error: si ocurre un error de conexion o al ejecutar el UPDATE

```
func (ctrl *AdminController) EditarProveedor(c *gin.Context)
    EditarProveedor
```

Actualiza el nombre, URL de API y porcentaje de ganancia de un proveedor. El ID del proveedor se lee desde el parametro de URL :id.

- HTTP 400 Bad Request: si el ID es invalido o el body no puede ser parseado

```
func (ctrl *AdminController) ListarProveedores(c *gin.Context)
    ListarProveedores
```

Retorna la lista completa de proveedores registrados con su tipo, estado, URL de API y porcentaje de ganancia configurado, ordenada por ID ascendente.

- HTTP 200 OK: JSON con arreglo de proveedores incluyendo campo activo derivado del EstadoID
- HTTP 500 Internal Server Error: si ocurre un error de conexion o consulta

```
func (ctrl *AdminController) ListarUsuarios(c *gin.Context)
    ListarUsuarios
```

Retorna la lista completa de usuarios registrados en el sistema junto con su rol asignado, ordenada por ID ascendente.

- HTTP 200 OK: JSON con arreglo de usuarios (id, nombre, apellido, correo, fechaRegistro, rolId, rol)

```
func (ctrl *AdminController) ObtenerMetricas(c *gin.Context)
    ObtenerMetricas
```

Retorna el desglose financiero detallado de todas las reservaciones, separando el monto cobrado, el costo base y la ganancia por cada detalle (vuelo u hotel) segun el porcentaje de ganancia configurado en cada proveedor. Tambien incluye un resumen global agrupado por tipo de reservacion.

- HTTP 200 OK: JSON con campos resumen (totales globales y por tipo) y reservaciones (lista detallada con desglose financiero individual)
- El costo base se calcula como: $\text{cobrado} / (1 + \text{porcentaje}/100)$
- La ganancia es la diferencia entre cobrado y base

```
func (ctrl *AdminController) ReservacionesRecientes(c *gin.Context)
    ReservacionesRecientes
```

Retorna las ultimas 10 reservaciones realizadas en la plataforma por cualquier usuario, ordenadas por fecha de creacion descendente. Incluye nombre del usuario, tipo, total y estado de cada reservacion.

- HTTP 200 OK: JSON con arreglo de hasta 10 reservaciones recientes

```
func (ctrl *AdminController) ToggleEstadoProveedor(c *gin.Context)
    ToggleEstadoProveedor
```

Activa o desactiva un proveedor segun el valor del campo activo recibido en el body. El ID del proveedor se lee desde el parametro de URL :id. EstadoID 1 equivale a activo, EstadoID 2 equivale a inactivo.

```
type AsientoVueloController struct {
    AsientoVueloController
```

Controlador encargado de gestionar el mapa de asientos de un vuelo y el cambio de asiento de un boleto especifico.

```
func NewAsientoVueloController(service *services.AsientoVueloService) *AsientoVueloController
    NewAsientoVueloController
```

Crea e inicializa un nuevo AsientoVueloController con el servicio recibido.

- service: instancia del servicio de asientos de vuelo
- *AsientoVueloController: puntero al controlador creado

```
func (ctrl *AsientoVueloController) CambiarAsiento(c *gin.Context)
    CambiarAsiento
```

Handler HTTP que actualiza el asiento de un boleto especifico dentro de una reservacion. Valida la sesion del usuario y delega la operacion al servicio, que verifica que el boleto pertenezca a la reserva del usuario.

- HTTP 200: mensaje de confirmacion indicando que el cambio fue procesado por la aerolinea
- HTTP 400: error si los datos enviados son invalidos o el servicio retorna un error
- HTTP 401: error si el usuario no esta autenticado

```
func (ctrl *AsientoVueloController) ObtenerAsientos(c *gin.Context)
    ObtenerAsientos
```

Handler HTTP que devuelve el mapa de asientos de un vuelo para el usuario autenticado. Valida la sesion del usuario y delega la consulta al servicio.

- HTTP 200: objeto AsientosVueloResponse con el mapa de asientos y boletos de la agencia
- HTTP 400: error si el body JSON es invalido o el servicio retorna un error

```
type BusquedaController struct {
    BusquedaController
```

Controlador encargado de gestionar las busquedas de vuelos y hoteles disponibles a traves de los proveedores registrados.

```
func NewBusquedaController(service *services.BusquedaService) *BusquedaController
    NewBusquedaController
```

Crea e inicializa un nuevo BusquedaController con el servicio recibido.

- service: instancia del servicio de busqueda
- *BusquedaController: puntero al controlador creado

```
func (ctrl *BusquedaController) BuscarHoteles(c *gin.Context)
```

BuscarHoteles

Handler HTTP que recibe los criterios de búsqueda de hoteles y retorna los resultados obtenidos de los proveedores hoteleros.

- HTTP 200: lista de hoteles disponibles que coinciden con los criterios
- Ruta esperada: POST /busqueda/hoteles

```
func (ctrl *BusquedaController) BuscarVuelos(c *gin.Context)
```

BuscarVuelos

Handler HTTP que recibe los criterios de búsqueda de vuelos y retorna los resultados obtenidos de los proveedores de aerolíneas.

- HTTP 200: lista de vuelos disponibles que coinciden con los criterios
- Ruta esperada: POST /busqueda/vuelos

```
type CancelacionController struct {
```

CancelacionController

Controlador encargado de gestionar la verificación y ejecución de cancelaciones de reservaciones realizadas por el usuario.

```
func NewCancelacionController(s *services.CancelacionService) *CancelacionController
```

NewCancelacionController

Crea e inicializa un nuevo CancelacionController con el servicio recibido.

- s: instancia del servicio de cancelacion
- *CancelacionController: puntero al controlador creado

```
func (ctrl *CancelacionController) Cancelar(c *gin.Context)
```

Cancelar

Handler HTTP que ejecuta la cancelacion de una reservacion existente.

El motivo de cancelacion es opcional; si no se proporciona se usa una cadena vacia.

- HTTP 200: mensaje confirmando que la reservacion fue cancelada exitosamente
 - HTTP 400: error si el parametro de ruta ID no es un entero valido
 - HTTP 422: error si el servicio no puede procesar la cancelacion
- Ruta esperada: POST /api/reservaciones/:id/cancelar

```
func (ctrl *CancelacionController) Verificar(c *gin.Context)
```

Verificar

Handler HTTP que verifica si una reservacion puede ser cancelada, devolviendo las condiciones y penalizaciones aplicables antes de confirmar la cancelacion.

- HTTP 200: resultado de la verificacion con condiciones de cancelacion
 - HTTP 404: error si la reservacion no existe o no pertenece al usuario
- Ruta esperada: GET /api/reservaciones/:id/cancelar/verificar

```
type CatalogoController struct {
```

CatalogoController

Controlador encargado de gestionar la actualizacion del catalogo de productos (vuelos, hoteles y paquetes) obtenidos desde los proveedores externos.

```
func NewCatalogoController(service *services.CatalogoService) *CatalogoController
```

NewCatalogoController

Crea e inicializa un nuevo CatalogoController con el servicio recibido.

- service: instancia del servicio de catalogo
- *CatalogoController: puntero al controlador creado

```
func (ctrl *CatalogoController) ActualizarCatalogo(c *gin.Context)
    ActualizarCatalogo
```

Handler HTTP que dispara el proceso de actualizacion del catalogo consultando los proveedores registrados y sincronizando la informacion en la base de datos.

- HTTP 200: mensaje de proceso completado junto con el detalle de resultados por proveedor
- HTTP 500: error interno si el servicio falla durante la actualizacion

```
type ComentarioController struct {
    ComentarioController
```

Controlador que maneja los endpoints de consulta de comentarios asociados a vuelos y hoteles de proveedores registrados.

```
func NewComentarioController(s *services.ComentarioService) *ComentarioController
    NewComentarioController
```

Constructor que retorna una nueva instancia de ComentarioController con el servicio de comentarios inyectado.

- s: puntero al servicio de comentarios
- *ComentarioController: puntero a la nueva instancia

```
func (ctrl *ComentarioController) ObtenerComentariosHotel(c *gin.Context)
    ObtenerComentariosHotel
```

Retorna la lista de comentarios asociados a un hotel especifico de un proveedor dado. Los parametros proveedorId y hotelId se leen desde la URL.

- HTTP 200 OK: JSON con la lista de comentarios del hotel
- HTTP 400 Bad Request: si proveedorId o hotelId no son enteros validos
- HTTP 500 Internal Server Error: si ocurre un error en la capa de servicio

```
func (ctrl *ComentarioController) ObtenerComentariosVuelo(c *gin.Context)
    ObtenerComentariosVuelo
```

Retorna la lista de comentarios asociados a una ruta de vuelo especifica de un proveedor dado. Los parametros proveedorId y rutaId se leen desde la URL.

- HTTP 200 OK: JSON con la lista de comentarios del vuelo
- HTTP 400 Bad Request: si proveedorId o rutaId no son enteros validos

```
type ConfiguracionController struct {  
    ConfiguracionController
```

Controlador encargado de exponer la configuracion global de la agencia, incluyendo el porcentaje de descuento aplicado a reservaciones de tipo paquete.

```
func NewConfiguracionController(db *sql.DB) *ConfiguracionController  
    NewConfiguracionController
```

Crea e inicializa un nuevo ConfiguracionController con la conexion a la base de datos.

- db: conexion activa a la base de datos
- *ConfiguracionController: puntero al controlador creado

```
func (ctrl *ConfiguracionController) ObtenerDescuento(c *gin.Context)  
    ObtenerDescuento
```

Handler HTTP publico que retorna el porcentaje de descuento configurado para reservaciones de tipo paquete. Si la consulta falla retorna 0.

- HTTP 200: JSON con el campo porcentaje_descuento
- Ruta esperada: GET /api/configuracion/descuento

```
type DetalleReservacionController struct {  
    DetalleReservacionController
```

Controlador encargado de agregar detalles a una reservacion existente, incluyendo vuelos con sus pasajeros y habitaciones de hotel.

```
func NewDetalleReservacionController(service *services.DetalleReservacionService) *DetalleReservacionController  
    NewDetalleReservacionController
```

Crea e inicializa un nuevo DetalleReservacionController con el servicio recibido.

- service: instancia del servicio de detalle de reservacion
- *DetalleReservacionController: puntero al controlador creado

```
func (ctrl *DetalleReservacionController) AgregarDetalleHotel(c *gin.Context)
    AgregarDetalleHotel
```

Handler HTTP que agrega un detalle de hotel a la reservacion del usuario autenticado. Deserializa la solicitud y delega la operacion al servicio.

- HTTP 200: respuesta del servicio con el detalle de hotel registrado

```
func (ctrl *DetalleReservacionController) AgregarDetalleVuelo(c *gin.Context)
    AgregarDetalleVuelo
```

Handler HTTP que agrega un detalle de vuelo a la reservacion del usuario autenticado. Valida la sesion y deserializa la solicitud antes de delegar al servicio.

- HTTP 200: respuesta del servicio con el detalle de vuelo registrado

```
func (ctrl *DetalleReservacionController) AgregarPasajerosVuelo(c *gin.Context)
    AgregarPasajerosVuelo
```

Handler HTTP que registra los datos de los pasajeros asociados a un vuelo dentro de la reservacion del usuario autenticado.

- HTTP 200: mensaje confirmando que los datos de pasajeros fueron guardados correctamente

```
type HandshakeController struct {
    HandshakeController
```

Controlador encargado de iniciar el proceso de handshake con proveedores de aerolineas, obteniendo el token de sesion necesario para consumir sus servicios.

```
func NewHandshakeController(service *services.HandshakeService) *HandshakeController
    NewHandshakeController
```

Crea e inicializa un nuevo HandshakeController con el servicio recibido.

- service: instancia del servicio de handshake de aerolineas
- *HandshakeController: puntero al controlador creado

```
func (ctrl *HandshakeController) IniciarHandshake(c *gin.Context)
    IniciarHandshake
```

Handler HTTP que inicia el proceso de handshake con un proveedor de aerolinea identificado por su ID en la ruta. Si el proceso es exitoso retorna el token de salida generado por el proveedor.

- HTTP 200: mensaje de exito junto con el token de salida del proveedor
- HTTP 400: error si el parametro de ruta ID no es un entero valido o el servicio retorna un error
- El parametro de ruta :id corresponde al ID del proveedor de aerolinea

```
type HandshakeHoteleraController struct {
    HandshakeHoteleraController
```

hoteleros, obteniendo el token de sesion necesario para consumir sus

```
func NewHandshakeHoteleraController(service *services.HandshakeHoteleraService) *HandshakeHoteleraController
    NewHandshakeHoteleraController
```

Crea e inicializa un nuevo HandshakeHoteleraController con el servicio

- service: instancia del servicio de handshake hotelero
- *HandshakeHoteleraController: puntero al controlador creado

```
func (ctrl *HandshakeHoteleraController) IniciarHandshake(c *gin.Context)
```

Handler HTTP que inicia el proceso de handshake con un proveedor hotelero identificado por su ID en la ruta. Si el proceso es exitoso retorna el token de salida generado por el proveedor.

hotelero

- El parametro de ruta :id corresponde al ID del proveedor hotelero

```
type LoginController struct {
    LoginController
```

Controlador encargado de gestionar la autenticacion de usuarios, incluyendo el inicio y cierre de sesion mediante JWT almacenado en cookie.

```
func NewLoginController(service *services.LoginService) *LoginController
    NewLoginController
```

Crea e inicializa un nuevo LoginController con el servicio recibido.

- service: instancia del servicio de login
- *LoginController: puntero al controlador creado

```
func (ctrl *LoginController) Login(c *gin.Context)
    Login
```

Handler HTTP que autentica al usuario con sus credenciales. Si son validas genera un token JWT y lo persiste en una cookie HttpOnly con duracion de 24 horas, retornando ademas los datos del usuario en el cuerpo de la respuesta.

- HTTP 200: datos del usuario autenticado y cookie de sesion establecida
- HTTP 400: error si el body JSON es invalido
- HTTP 401: error si las credenciales son incorrectas
- HTTP 500: error interno al generar el token JWT o al procesar el login

```
func (ctrl *LoginController) Logout(c *gin.Context)
    Logout
```

Handler HTTP que cierra la sesion del usuario eliminando la cookie de sesion al establecer su tiempo de vida en -1.

- HTTP 200: mensaje confirmando que la sesion fue cerrada

```
type MisReservacionesController struct {
    MisReservacionesController
```

Controlador encargado de exponer las reservaciones del usuario autenticado, tanto en formato de listado resumido como en detalle completo consultando a los proveedores externos.

```
func NewMisReservacionesController(s *services.MisReservacionesService) *MisReservacionesController
    NewMisReservacionesController
```

Crea e inicializa un nuevo MisReservacionesController con el servicio

- s: instancia del servicio de mis reservaciones
- *MisReservacionesController: puntero al controlador creado

```
func (ctrl *MisReservacionesController) Detalle(c *gin.Context)
```

Detalle

Handler HTTP que devuelve el detalle completo de una reservacion especifica del usuario autenticado, consultando informacion actualizada de los proveedores externos.

- HTTP 200: objeto con el detalle completo de la reservacion
- Ruta esperada: GET /api/reservaciones/mias/:id

```
func (ctrl *MisReservacionesController) Listar(c *gin.Context)
```

Listar

Handler HTTP que devuelve todas las reservaciones del usuario autenticado con los datos almacenados localmente en la base de datos de la agencia.

- HTTP 200: lista de reservaciones del usuario
- HTTP 500: error interno si el servicio falla al consultar las reservaciones
- Ruta esperada: GET /api/reservaciones/mias

```
type PagoController struct {
```

PagoController

Controlador que maneja los endpoints relacionados al procesamiento de pagos de reservaciones existentes en la plataforma.

```
func NewPagoController(s *services.PagoService) *PagoController
```

NewPagoController

Constructor que retorna una nueva instancia de PagoController con el servicio de pagos inyectado.

- s: puntero al servicio de pagos
- *PagoController: puntero a la nueva instancia

```
func (ctrl *PagoController) Pagar(c *gin.Context)
```

```
    Pagar
```

Procesa el pago de una reservacion existente para el usuario autenticado. Lee el ID del usuario desde el contexto de Gin, valida el body del request y delega el procesamiento al servicio de pagos.

- HTTP 200 OK: JSON con mensaje de confirmacion si el pago fue exitoso
- HTTP 400 Bad Request: si los datos de pago en el body estan incompletos o son invalidos
- HTTP 422 Unprocessable Entity: si el servicio rechaza el pago (fondos insuficientes, reservacion no encontrada, etc.)
- El ID del usuario autenticado se obtiene del contexto sin verificacion de existencia

```
type PerfilController struct {
```

```
    PerfilController
```

Controlador que maneja los endpoints de consulta y actualizacion del perfil del usuario autenticado, incluyendo telefono y contrasena.

```
func NewPerfilController(service *services.PerfilService) *PerfilController
```

```
    NewPerfilController
```

Constructor que retorna una nueva instancia de PerfilController con el servicio de perfil inyectado.

- service: puntero al servicio de perfil
- *PerfilController: puntero a la nueva instancia

```
func (ctrl *PerfilController) ActualizarTelefono(c *gin.Context)
```

```
    ActualizarTelefono
```

Actualiza el numero de telefono del usuario autenticado. El nuevo numero se lee del campo telefono en el body JSON de la solicitud.

- HTTP 400 Bad Request: si el campo telefono no esta presente en el body
- HTTP 500 Internal Server Error: si ocurre un error al actualizar el telefono

```
func (ctrl *PerfilController) CambiarContrasena(c *gin.Context)
```

```
    CambiarContrasena
```

Cambia la contraseña del usuario autenticado. Verifica que la contraseña actual sea correcta, que la nueva y su confirmación coincidan, y que la nueva tenga al menos 8 caracteres antes de persistir el cambio.

- HTTP 400 Bad Request: si faltan campos, las contraseñas no coinciden o la nueva es muy corta
- HTTP 401 Unauthorized: si la contraseña actual proporcionada es incorrecta
- HTTP 500 Internal Server Error: si ocurre un error al verificar o cambiar la contraseña
- La verificación de la contraseña actual se realiza comparando el hash almacenado

```
func (ctrl *PerfilController) ObtenerPerfil(c *gin.Context)
```

```
    ObtenerPerfil
```

Retorna los datos del perfil del usuario actualmente autenticado. El ID del usuario se extrae del contexto de Gin inyectado por el middleware.

- HTTP 200 OK: JSON con los datos del perfil del usuario
- HTTP 500 Internal Server Error: si ocurre un error al obtener el perfil

```
type ProveedorController struct {
```

```
    ProveedorController
```

Controlador encargado de gestionar el alta de proveedores externos (aerolíneas y hoteles) en el sistema de la agencia.

```
func NewProveedorController(service *services.ProveedorService) *ProveedorController
```

```
    NewProveedorController
```

Crea e inicializa un nuevo ProveedorController con el servicio recibido.

- service: instancia del servicio de proveedor
- *ProveedorController: puntero al controlador creado

```
func (ctrl *ProveedorController) CrearProveedor(c *gin.Context)
    CrearProveedor
```

Handler HTTP que registra un nuevo proveedor externo en el sistema a partir de los datos enviados en el body de la solicitud.

- HTTP 201: mensaje de confirmacion junto con los datos del proveedor creado
- error de validacion

```
type ReservacionController struct {
    ReservacionController
```

Controlador que maneja los endpoints de creacion de reservaciones, descarga de PDF y envio de correo de confirmacion al usuario autenticado.

```
func NewReservacionController(
    service *services.ReservacionService,
    pdfService *services.PdfReservacionService,
    emailService *services.EmailReservacionService,
) *ReservacionController
    NewReservacionController
```

Constructor que retorna una nueva instancia de ReservacionController con los servicios de reservacion, PDF y correo inyectados.

- service: servicio principal de reservaciones
- pdfService: servicio de generacion de PDF
- emailService: servicio de envio de correo de confirmacion
- *ReservacionController: puntero a la nueva instancia

```
func (ctrl *ReservacionController) CrearReservacion(c *gin.Context)
    CrearReservacion
```

Crea una nueva reservacion para el usuario autenticado. Valida que el tipo de reserva sea 1 (Aerolinea), 2 (Hotelera) o 3 (Paquete) antes de delegar al

- HTTP 201 Created: JSON con los datos de la reservacion creada
- HTTP 400 Bad Request: si el body es invalido o el tipo de reserva es incorrecto
- HTTP 401 Unauthorized: si el usuario no esta autenticado

```
func (ctrl *ReservacionController) DescargarPDF(c *gin.Context)
```

DescargarPDF

Genera y retorna el PDF de una reservacion especifica del usuario autenticado. El ID de la reservacion se lee desde el parametro de URL :id.

- HTTP 200 OK: archivo PDF adjunto con nombre reservacion-{id}.pdf
- HTTP 400 Bad Request: si el ID de la reservacion no es un entero valido
- HTTP 404 Not Found: si la reservacion no existe
- HTTP 500 Internal Server Error: si ocurre un error al generar el PDF
- El PDF se entrega con Content-Disposition attachment para forzar la descarga

```
func (ctrl *ReservacionController) EnviarCorreo(c *gin.Context)
```

EnviarCorreo

Envía un correo de confirmacion al usuario autenticado para una reservacion especifica. El ID de la reservacion se lee desde el parametro de URL :id.

- HTTP 200 OK: JSON con mensaje de exito al enviar el correo
- Los errores del servicio de correo se registran en el log del servidor

```
type SesionController struct{}
```

SesionController

Controlador que maneja los endpoints relacionados a la sesion del usuario autenticado.

```
func NewSesionController() *SesionController
```

NewSesionController

Constructor que retorna una nueva instancia de SesionController.

- *SesionController: puntero a la nueva instancia

```
func (ctrl *SesionController) ObtenerSesion(c *gin.Context)
```

ObtenerSesion

Retorna los datos de la sesion del usuario actualmente autenticado, extrayendo el ID, nombre de usuario y rol desde el contexto de Gin inyectado por el middleware de autenticacion.

- HTTP 200 OK: JSON con campos usuario_id, username y rol_id

```
type StatsController struct {  
    StatsController
```

Controlador que maneja los endpoints de estadísticas generales de la plataforma, incluyendo conteos de proveedores, usuarios, reservaciones e ingresos totales. Es de acceso público, no requiere autenticación.

```
func NewStatsController(db *sql.DB) *StatsController  
    NewStatsController
```

Constructor que retorna una nueva instancia de StatsController con la

- *StatsController: puntero a la nueva instancia

```
func (ctrl *StatsController) ObtenerStats(c *gin.Context)  
    ObtenerStats
```

Retorna un resumen estadístico de la plataforma consultando directamente la base de datos. Incluye conteos de aerolíneas, hoteles, usuarios, reservaciones por estado y por tipo, así como los ingresos totales.

- HTTP 200 OK: JSON con todas las estadísticas de la plataforma
- HTTP 500 Internal Server Error: si ocurre un error de conexión a la base de datos
- Este endpoint es público y no requiere autenticación
- Los ingresos totales se calculan sumando el campo Total de detalles_reservacion

```
type UsuarioController struct {  
    UsuarioController
```

Controlador que maneja los endpoints de gestión de usuarios, incluyendo el registro de nuevas cuentas y la consulta del listado completo para el panel de administración.

```
func NewUsuarioController(service *services.UsuarioService) *UsuarioController  
    NewUsuarioController
```

Constructor que retorna una nueva instancia de UsuarioController con el servicio de usuario inyectado.

- service: puntero al servicio de usuario
- *UsuarioController: puntero a la nueva instancia

```
func (ctrl *UsuarioController) ObtenerTodos(c *gin.Context)
```

ObtenerTodos

Retorna la lista completa de usuarios registrados en el sistema con sus datos basicos y rol asignado. Usado por el panel de administracion para gestion de roles y asignacion de usuarios WebService a proveedores.

- HTTP 200 OK: JSON con el listado de usuarios
- HTTP 500 Internal Server Error: si ocurre un error al consultar la base

```
func (ctrl *UsuarioController) Registrar(c *gin.Context)
```

Registrar

Registra un nuevo usuario en el sistema a partir de los datos enviados en el body de la solicitud. Verifica duplicados de correo, pasaporte y username. Si el registro es exitoso, envia un correo de bienvenida en segundo plano sin bloquear la respuesta al cliente.

- HTTP 201 Created: JSON con mensaje de exito si el usuario fue registrado
- HTTP 400 Bad Request: si el body no puede ser parseado o tiene datos invalidos
- HTTP 409 Conflict: JSON con campos duplicados (correo, pasaporte, username)
- HTTP 500 Internal Server Error: si ocurre un error interno al registrar
- El correo de bienvenida se envia de forma asincrona (goroutine fire-and-forget)
- Los errores del envio de correo se registran en el log del servidor

```
type WebserviceController struct {
```

WebserviceController

Controlador que maneja los endpoints del webservice externo, permitiendo a proveedores autenticados notificar cambios de estado sobre reservaciones mediante un token de proveedor.

```
func NewWebserviceController(db *sql.DB) *WebserviceController
```

NewWebserviceController

Constructor que retorna una nueva instancia de WebserviceController con la

- *WebserviceController: puntero a la nueva instancia

```
func (ctrl *WebserviceController) RecibirNotificacion(c *gin.Context)
```

```
    RecibirNotificacion
```

Recibe una notificacion de cambio de estado desde un proveedor externo autenticado via header X-Proveedor-Token. Busca la reservacion interna usando el ID de reserva del proveedor y actualiza su estado en la base de datos. Si el nuevo estado es cancelada, guarda el motivo en el campo parametros_json del detalle de la reservacion.

- HTTP 200 OK: JSON con mensaje de confirmacion, reservacion_id y nuevo_estado
- HTTP 400 Bad Request: si el body es invalido, el estado no es reconocido, o falta el motivo en una cancelacion
- HTTP 404 Not Found: si no se encuentra la reservacion para el proveedor indicado
actualizar
- La identidad del proveedor (proveedor_id) es inyectada por el middleware ProveedorRequerido
- Los estados validos son: cancelada, confirmada, completada, en curso
- El motivo es obligatorio cuando nuevoEstado es cancelada

Services

Servicios de negocio de la agencia de viajes. Este paquete contiene la logica central para reservaciones, busquedas, autenticacion, catalogos y comunicacion con proveedores externos (aerolineas y hoteleras).

Contiene los servicios de negocio de la agencia de viajes, incluyendo procesamiento de pagos, reservaciones, proveedores y usuarios.

CONSTANTS

```
const (
    TipoAerolinea = 1
    TipoHotelera  = 2

    TipoDetalleVuelo = 1
    TipoDetalleHotel = 2

    TipoReservaAerolinea = 1
    TipoReservaHotelera  = 2
    TipoReservaPaquete   = 3
)
```

VARIABLES

```
var (
    ErrUsuarioNoEncontrado = errors.New("usuario no encontrado")
    ErrContrasenaInvalida  = errors.New("contrase??a inv?ilida")

    var ErrCredencialesInvalidas = errors.New("credenciales inv?ilidas")
)
```

TYPES

```
type AsientoVueloService struct {
```

```
    // Has unexported fields.
```

```
}
```

```
    AsientoVueloService
```

```
    Servicio encargado de gestionar los asientos de vuelo asociados a
    reservaciones de aerolinea. Permite consultar los asientos disponibles y
    realizar cambios de asiento para boletos especificos de una reservacion.
```

```
func NewAsientoVueloService(db *sql.DB) *AsientoVueloService
    NewAsientoVueloService
```

Crea e inicializa una nueva instancia de AsientoVueloService con su repositorio de detalle de reservacion.

Parametros:

- db: conexion activa a la base de datos SQL

Retorna:

- *AsientoVueloService: instancia lista para usar

```
func (s *AsientoVueloService) CambiarAsientoVuelo(
    usuarioID int,
    req dto.CambiarAsientoVueloRequest,
) error
    CambiarAsientoVuelo
```

Cambia el asiento asignado a un boleto especifico de una reservacion de vuelo. Valida que el boleto pertenezca a la reservacion del usuario antes de enviar la solicitud de cambio al proveedor aerolinea.

- usuarioID: identificador del usuario dueño de la reservacion
- req: datos del cambio incluyendo ReservacionID, ProveedorID, BoletoID y NuevoAsiento
- error: si el boleto no pertenece a la reservacion o falla la API del proveedor

```
func (s *AsientoVueloService) ObtenerAsientosVuelo(
    req dto.ObtenerAsientosVueloRequest,
) (*dto.AsientosVueloResponse, error)
    ObtenerAsientosVuelo
```

Consulta los asientos de vuelo disponibles para una reservacion especifica, obteniendo primero los datos de conexion del proveedor aerolinea desde la BD y luego llamando a su API externa.

- req: datos de la solicitud incluyendo ReservacionID y ProveedorID
- *dto.AsientosVueloResponse: lista de vuelos con sus boletos y asientos
- error: si la reservacion no existe, no pertenece al usuario o falla la API del proveedor

```
type BusquedaService struct {
    BusquedaService
```

Servicio encargado de realizar busquedas de vuelos y hoteles consultando los proveedores externos registrados en el catalogo. Aplica el margen de ganancia configurado por proveedor sobre los precios retornados.

```
func NewBusquedaService(db *sql.DB) *BusquedaService
    NewBusquedaService
```

Crea e inicializa una nueva instancia de BusquedaService con su repositorio de busqueda.

- *BusquedaService: instancia lista para usar

```
func (s *BusquedaService) BuscarHoteles(req dto.BusquedaHotelesRequest) ([]dto.BusquedaHotelesResponse, error)
    BuscarHoteles
```

Busca hoteles disponibles en una ciudad consultando todos los proveedores hoteleros registrados para esa ubicacion en el catalogo. Resuelve el ID de ciudad, obtiene la lista de proveedores de tipo hotelera y llama a cada uno de forma individual. Si un proveedor falla, se incluye su error en la respuesta sin interrumpir las demas consultas.

- req: datos de busqueda incluyendo ciudad, pais y demas filtros de hospedaje
- []dto.BusquedaHotelesResponse: lista de respuestas por proveedor, con datos o error
- error: si falla la resolucion de ciudad o la consulta de proveedores en BD

```
func (s *BusquedaService) BuscarVuelos(req dto.BusquedaVuelosRequest) ([]dto.BusquedaVuelosResponse, error)
    BuscarVuelos
```

Busca vuelos disponibles entre dos ciudades consultando todos los proveedores aerolineas registrados para esa ruta en el catalogo. Resuelve los IDs de ciudad para origen y destino, obtiene la lista de proveedores activos y llama a cada uno de forma individual. Si un proveedor falla, se incluye su error en la respuesta sin interrumpir las demas consultas.

- req: datos de busqueda incluyendo ciudad/pais de origen, destino y demas filtros
- []dto.BusquedaVuelosResponse: lista de respuestas por proveedor,
- error: si falla la resolucion de ciudades o la consulta de proveedores en BD

```
type CancelacionService struct {
    CancelacionService
```

Servicio encargado de gestionar el proceso de cancelacion de reservaciones. Implementa un flujo de dos pasos: primero verifica si todos los proveedores involucrados permiten la cancelacion, y luego ejecuta la cancelacion en cada proveedor y en la base de datos local de forma atomica (todo o nada).

```
func NewCancelacionService(repo *repositories.CancelacionRepository) *CancelacionService
NewCancelacionService
```

Crea e inicializa una nueva instancia de CancelacionService con el repositorio de cancelacion proporcionado.

- repo: repositorio de cancelacion ya inicializado
- *CancelacionService: instancia lista para usar

```
func (s *CancelacionService) CancelarReservacion(reservacionID, usuarioID int, motivo string) error
CancelarReservacion
```

Paso 2 del flujo de cancelacion. Ejecuta la cancelacion completa de una reservacion. Primero verifica el estado y re-valida con cada proveedor (rollback logico), luego cancela en cada proveedor externo y finalmente actualiza el estado en la base de datos local. Si algun proveedor rechaza la cancelacion, el proceso se detiene sin cancelar nada.

- reservacionID: identificador de la reservacion a cancelar
- motivo: descripcion del motivo de cancelacion
- error: si la reservacion no es cancelable, algun proveedor rechaza o falla la BD

```
func (s *CancelacionService) VerificarCancelacion(reservacionID, usuarioID int)
(*dto.VerificarCancelacionResponse, error)
VerificarCancelacion
```

Paso 1 del flujo de cancelacion. Verifica si una reservacion puede ser cancelada consultando el estado local y luego preguntando a cada proveedor externo involucrado. Si cualquier proveedor rechaza la cancelacion, el resultado global indica que no es posible cancelar.

- reservacionID: identificador de la reservacion a verificar
- *dto.VerificarCancelacionResponse: resultado con flag global y detalle por proveedor

```
type CatalogoSchedulService struct {
CatalogoSchedulService
```

Servicio encargado de actualizar automaticamente el catalogo de la agencia cada 7 dias en segundo plano. Utiliza el CatalogoService para ejecutar la sincronizacion con todos los proveedores activos registrados en BD. El proceso puede detenerse de forma controlada mediante Detener.

```
func NewCatalogoSchedulService(service *CatalogoService) *CatalogoSchedulService
    NewCatalogoSchedulService
```

Crea e inicializa una nueva instancia de CatalogoSchedulService con el servicio de catalogo necesario para ejecutar la actualizacion y el canal de control para detener el proceso en segundo plano.

- service: instancia del servicio de catalogo a utilizar en cada ciclo
- *CatalogoSchedulService: instancia lista para usar, aun no iniciada

```
func (s *CatalogoSchedulService) Detener()
    Detener
```

Detiene el proceso de actualizacion automatica en segundo plano cerrando el canal de control. Debe llamarse al apagar la aplicacion para liberar la goroutine.

```
func (s *CatalogoSchedulService) Iniciar()
    Iniciar
```

Lanza en segundo plano una goroutine que actualiza el catalogo completo de la agencia cada 7 dias llamando al CatalogoService. El proceso puede detenerse llamando a Detener.

Notas:

- Registra en log cada actualizacion completada, el numero de proveedores procesados o cualquier error ocurrido durante el ciclo

```
type CatalogoService struct {
    CatalogoService
```

Servicio encargado de sincronizar el catalogo local de rutas y ubicaciones con la informacion provista por los proveedores externos registrados. Soporta proveedores de tipo aerolinea (rutas origen-destino) y hotelera (hoteles por ciudad), gestionando la creacion de ubicaciones nuevas mediante el UbicacionService.

```
func NewCatalogoService(db *sql.DB, ubicacionService *UbicacionService) *CatalogoService
    NewCatalogoService
```

Crea e inicializa una nueva instancia de CatalogoService con su repositorio y el servicio de ubicaciones requerido.

- ubicacionService: servicio para obtener o crear ciudades/paises en BD
- *CatalogoService: instancia lista para usar

```
func (s *CatalogoService) ActualizarCatalogo() ([]dto.ActualizarCatalogoResponse, error)
    ActualizarCatalogo
```

Actualiza el catalogo completo de la agencia iterando sobre todos los proveedores activos registrados en BD. Por cada proveedor llama a actualizarProveedor y acumula el resultado o el error en la respuesta. Un fallo en un proveedor no detiene el proceso para los demas.

- []dto.ActualizarCatalogoResponse: lista de resultados por proveedor con conteo de insertados
- error: si falla la consulta de proveedores activos en BD

```
type ComentarioService struct {
    ComentarioService
```

Servicio encargado de obtener comentarios de vuelos y hoteles consultando directamente las APIs de los proveedores externos registrados.

```
func NewComentarioService(proveedorRepo *repositories.ProveedorRepository) *ComentarioService
    NewComentarioService
```

Crea e inicializa una nueva instancia de ComentarioService con su repositorio de proveedores.

- proveedorRepo: repositorio de proveedores para obtener URL y token de acceso
- *ComentarioService: instancia inicializada del servicio de comentarios

```
func (s *ComentarioService) ObtenerComentariosHotel(proveedorID, hotelID int) (interface{}, error)
    ObtenerComentariosHotel
```

Obtiene los comentarios de un hotel consultando la API del proveedor identificado por su ID. Construye la URL del endpoint de comentarios de hotel y delega la llamada HTTP al metodo interno llamarProveedor.

- proveedorID: identificador del proveedor hotelero en la base de datos
- hotelID: identificador del hotel en el sistema del proveedor
- interface{}: respuesta JSON deserializada del proveedor
- error: error si el proveedor no existe o la llamada HTTP falla

```
func (s *ComentarioService) ObtenerComentariosVuelo(proveedorID, rutaID int) (interface{}, error)
    ObtenerComentariosVuelo
```

Obtiene los comentarios de una ruta de vuelo consultando la API del proveedor identificado por su ID. Construye la URL del endpoint de comentarios de aerolinea y delega la llamada HTTP al metodo interno llamarProveedor.

- proveedorID: identificador del proveedor de aerolinea en la base de datos
- rutaID: identificador de la ruta de vuelo en el sistema del proveedor

```
type DetalleReservacionService struct {
    DetalleReservacionService
```

Servicio encargado de agregar detalles de vuelo y hotel a reservaciones existentes en estado pendiente. Coordina la reserva con el proveedor externo, aplica el margen de ganancia configurado, almacena el detalle en BD y recalcula el total de la reservacion. Tambien gestiona el alta de pasajeros en el sistema de la aerolinea.

```
func NewDetalleReservacionService(db *sql.DB) *DetalleReservacionService
    NewDetalleReservacionService
```

Crea e inicializa una nueva instancia de DetalleReservacionService con su

- *DetalleReservacionService: instancia lista para usar

```
func (s *DetalleReservacionService) AgregarDetalleHotel(usuarioID int, req dto.AgregarDetalleHotelRequest)
(interface{}, error)
    AgregarDetalleHotel
```

Agrega un detalle de hotel a una reservacion existente. Valida que la reservacion pertenezca al usuario, este en estado pendiente y no sea exclusivamente de tipo aerea. Llama al proveedor hotelera para crear la reserva, calcula el precio con ganancia por noche (con soporte para personas extra) y guarda el detalle en BD, actualizando el total.

- req: datos del detalle incluyendo ReservacionID, ProveedorID y lista de habitaciones
- interface{}: mapa con mensaje, IDs, total base, total con ganancia y detalle del proveedor
- error: si la reservacion no existe, no es valida, falla el proveedor o la BD

```
func (s *DetalleReservacionService) AgregarDetalleVuelo(usuarioID int, req dto.AgregarDetalleVueloRequest) (interface{},
AgregarDetalleVuelo
```

Agrega un detalle de vuelo a una reservacion existente. Valida que la exclusivamente de tipo hotelera. Llama al proveedor aerolinea para crear la reserva, calcula el precio con ganancia por boleto y guarda el detalle en BD, actualizando el total de la reservacion.

vuelos

```
func (s *DetalleReservacionService) AgregarPasajerosVuelo(
    req dto.AgregarPasajerosVueloRequest,
    AgregarPasajerosVuelo
```

Registra los datos de los pasajeros en el sistema de la aerolinea para una reservacion existente. Valida que cada pasajero tenga un numero de pasaporte con solo digitos antes de enviar la solicitud al proveedor.

- req: datos de la solicitud incluyendo ReservacionID, ProveedorID y lista de pasajeros
- error: si el pasaporte es invalido, falla la obtencion del detalle o falla el proveedor

```
type EmailReservacionService struct {
    EmailReservacionService
```

Servicio encargado de generar el PDF de confirmacion de una reservacion y enviarlo al correo electronico del usuario titular.

```
func NewEmailReservacionService(
    misSvc *MisReservacionesService,
    pdfSvc *PdfReservacionService,
    usuRepo *repositories.UsuarioRepository,
) *EmailReservacionService
    NewEmailReservacionService
```

Crea e inicializa una nueva instancia de EmailReservacionService con sus dependencias.

- misSvc: servicio de mis reservaciones para obtener el detalle completo desde proveedores
- pdfSvc: servicio de PDF para generar el archivo adjunto
- usuRepo: repositorio de usuarios para obtener nombre y correo cuando no los provee el proveedor
- *EmailReservacionService: instancia inicializada del servicio de envio de correos

```
func (s *EmailReservacionService) EnviarConfirmacion(reservacionID, usuarioID int) error
    EnviarConfirmacion
```

Ejecuta el flujo completo de envio del correo de confirmacion: obtiene el detalle de la reservacion desde los proveedores, prepara los datos del PDF, completa nombre y correo del usuario desde la base de datos si es necesario (caso reservas de hotel puro), genera el PDF como adjunto, construye el cuerpo HTML del correo y lo envia al destinatario.

- reservacionID: identificador de la reservacion a confirmar
- usuarioID: identificador del usuario propietario de la reservacion
- error: error si la reservacion no existe, no se puede determinar el correo destinatario, falla la generacion del PDF o el envio del correo

```
type ExpiracionService struct {
    ExpiracionService
```

Servicio encargado de expirar automaticamente las reservaciones pendientes que han superado su tiempo límite. Ejecuta una revision periodica en segundo plano cada minuto y tambien permite expirar manualmente las reservaciones pendientes de un usuario especifico. Notifica a cada proveedor externo antes de actualizar el estado en la base de datos local.

```
func NewExpiracionService(db *sql.DB) *ExpiracionService
    NewExpiracionService
```

Crea e inicializa una nueva instancia de ExpiracionService con su repositorio de reservaciones y el canal de control para detener el proceso en segundo plano.

- *ExpiracionService: instancia lista para usar, aun no iniciada

```
func (s *ExpiracionService) Detener()
```

Detiene el proceso de expiracion en segundo plano cerrando el canal de control. Debe llamarse al apagar la aplicacion para liberar la goroutine.

```
func (s *ExpiracionService) ExpirarReservacionesDeUsuario(usuarioID int) error
    ExpirarReservacionesDeUsuario
```

Expira todas las reservaciones pendientes de un usuario específico que hayan superado su tiempo límite. Itera sobre las reservaciones obtenidas y llama a `expirarUna` para cada una, registrando en log los errores individuales sin interrumpir el proceso.

- `usuarioID`: identificador del usuario cuyas reservaciones se deben revisar
- `error`: si falla la consulta de reservaciones pendientes del usuario en

```
func (s *ExpiracionService) Iniciar()
```

Lanza en segundo plano una goroutine que revisa y expira reservaciones pendientes vencidas cada minuto. El proceso puede detenerse llamando a `Detener`.

- Registra en log cada revisión completada o error ocurrido

```
type HandshakeHoteleraService struct {
    HandshakeHoteleraService
```

Servicio encargado de gestionar el proceso de handshake de autenticación con proveedores de tipo hotelera. Genera un token de entrada para la agencia, lo envía a la hotelera y almacena ambos tokens (entrada y salida) en BD para su uso en comunicaciones posteriores.

```
func NewHandshakeHoteleraService(db *sql.DB, cfg *config.Config) *HandshakeHoteleraService
    NewHandshakeHoteleraService
```

Crea e inicializa una nueva instancia de `HandshakeHoteleraService` con su repositorio de proveedores y la URL pública de la agencia.

- `cfg`: configuración de la aplicación que contiene la URL del servidor
- `*HandshakeHoteleraService`: instancia lista para usar

```
func (s *HandshakeHoteleraService) IniciarHandshake(proveedorID int) (string, error)
    IniciarHandshake
```

Ejecuta el flujo completo de handshake con una hotelera proveedora.
Obtiene la URL del proveedor, genera un token de entrada para la agencia,
lo envia a la hotelera junto con la URL de la agencia, recibe el token de
salida del proveedor y guarda ambos tokens en BD.

- proveedorID: identificador del proveedor hotelera con quien hacer handshake
- string: token de salida recibido del proveedor hotelera
- error: si el proveedor no existe, no tiene URL, falla la generacion del token, falla la comunicacion con la hotelera o falla el guardado en BD

```
type HandshakeService struct {
    HandshakeService
```

proveedores de tipo aerolinea. Genera un token de entrada para la agencia,
lo envia a la aerolinea y almacena ambos tokens (entrada y salida) en BD
para su uso en comunicaciones posteriores.

```
func NewHandshakeService(db *sql.DB, cfg *config.Config) *HandshakeService
    NewHandshakeService
```

Crea e inicializa una nueva instancia de HandshakeService con su repositorio
de proveedores y la URL publica de la agencia.

- *HandshakeService: instancia lista para usar

```
func (s *HandshakeService) IniciarHandshake(proveedorID int) (string, error)
```

Ejecuta el flujo completo de handshake con una aerolinea proveedora.
lo envia a la aerolinea junto con la URL de la agencia, recibe el token de

- proveedorID: identificador del proveedor aerolinea con quien hacer
- string: token de salida recibido del proveedor aerolinea
token, falla la comunicacion con la aerolinea o falla el guardado en BD

```
type LoginService struct {
    LoginService
```

Servicio encargado de gestionar la autenticacion de usuarios en la agencia
de viajes. Valida las credenciales ingresadas contra la base de datos y
retorna la informacion del usuario autenticado para generar la sesion.

```
func NewLoginService(db *sql.DB) *LoginService
    NewLoginService
```

Crea e inicializa una nueva instancia de LoginService con su repositorio de login.

- *LoginService: instancia lista para usar

```
func (s *LoginService) Login(req dto.LoginRequest) (dto.LoginResponse, error)
    Login
```

Autentica a un usuario verificando sus credenciales contra la base de datos. Busca al usuario por username o correo electronico y verifica la contraseña usando hashing seguro. Si las credenciales son invalidas retorna un error generico para evitar revelar si el usuario existe o no.

- req: datos de login con campo Login (username o correo) y Contraseña
- dto.LoginResponse: datos del usuario autenticado (ID, nombre, apellido, correo, username, rol)
- error: ErrCredencialesInvalidas si el usuario no existe o la contraseña es incorrecta

```
type MisReservacionesService struct {
    MisReservacionesService
```

Servicio encargado de consultar y presentar las reservaciones de un usuario. Ofrece dos modos de consulta: un listado resumido con datos locales unicamente, y un detalle completo que enriquece cada reservacion consultando en tiempo real a los proveedores externos involucrados (aerolíneas y hoteleras).

```
func NewMisReservacionesService(repo *repositories.MisReservacionesRepository) *MisReservacionesService
    NewMisReservacionesService
```

Crea e inicializa una nueva instancia de MisReservacionesService con el repositorio de reservaciones proporcionado.

- repo: repositorio de mis reservaciones ya inicializado
- *MisReservacionesService: instancia lista para usar

```
func (s *MisReservacionesService) ListarReservaciones(usuarioID int) ([]dto.ReservacionResumenResponse, error)
    ListarReservaciones
```

Retorna el listado resumido de todas las reservaciones del usuario con sus detalles basicos. Agrupa las filas retornadas por la BD en reservaciones, organizando los detalles de cada una en orden de insercion.

- usuarioID: identificador del usuario cuyas reservaciones se desean listar
- []dto.ReservacionResumenResponse: lista de reservaciones con detalles resumidos
- error: si falla la consulta de reservaciones en BD

```
func (s *MisReservacionesService) ObtenerDetalle(reservacionID, usuarioID int)
(*dto.ReservacionDetalladaResponse, error)
    ObtenerDetalle
```

Retorna el detalle completo de una reservacion especifica del usuario, enriqueciendo cada detalle con la informacion en tiempo real obtenida del proveedor externo correspondiente. Si la consulta a un proveedor falla, el campo DataProveedor incluye el mensaje de error en lugar de los datos.

- reservacionID: identificador de la reservacion a consultar
- *dto.ReservacionDetalladaResponse: reservacion con detalles completos y datos de proveedores
- error: si la reservacion no existe o no pertenece al usuario

```
type PagoService struct {
    PagoService
```

Servicio encargado de gestionar el procesamiento de pagos de reservaciones, incluyendo validacion de tarjetas, verificacion de integridad de detalles, aplicacion de descuento por paquete y notificacion a proveedores externos.

```
func NewPagoService(
    repo *repositories.PagoRepository,
    rr *repositories.ReservacionRepository,
    cr *repositories.AgenciaConfiguracionRepository,
) *PagoService
    NewPagoService
```

Crea e inicializa una nueva instancia de PagoService con sus dependencias.

- repo: repositorio de pagos para operaciones en base de datos
- rr: repositorio de reservaciones para consultar detalles
- cr: repositorio de configuracion de la agencia para leer el descuento de paquetes
- *PagoService: instancia inicializada del servicio de pagos

```
func (s *PagoService) ProcesarPago(usuarioID int, req dto.PagoReservacionRequest) error
    ProcesarPago
```

Ejecuta el flujo completo de pago de una reservacion: valida los datos de la tarjeta, verifica que la reserva pertenezca al usuario y este pendiente, valida la integridad de los detalles segun el tipo de reserva, notifica a cada proveedor externo y finalmente confirma la reserva en la base de datos. Si el tipo de reserva es paquete (3), aplica el porcentaje de descuento configurado en Agencia_Configuracion sobre el total final. El descuento es absorbido por la agencia, no por los proveedores.

- usuarioID: identificador del usuario que realiza el pago
- req: datos del pago incluyendo numero de tarjeta, CVV, NIT y codigo postal
- error: error si la tarjeta es invalida, la reserva no existe o ya fue pagada, si los detalles no cumplen la estructura del tipo de reserva, o si algun proveedor rechaza el pago

```
type PdfReservacionService struct {
    PdfReservacionService
```

Servicio encargado de generar el PDF de una reservacion combinando los datos obtenidos desde los proveedores externos con la informacion del usuario almacenada en la base de datos propia.

```
func NewPdfReservacionService(
) *PdfReservacionService
    NewPdfReservacionService
```

Crea e inicializa una nueva instancia de PdfReservacionService con sus

- usuRepo: repositorio de usuarios para consultar nombre y correo cuando no los provee la aerolínea
- *PdfReservacionService: instancia inicializada del servicio de generación de PDF

```
func (s *PdfReservacionService) GenerarPDF(reservacionID, usuarioID int) ([]byte, error)
    GenerarPDF
```

Obtiene el detalle completo de una reservación desde los proveedores, lo mapea a la estructura de datos del PDF, completa el nombre y correo del usuario desde la base de datos si no fueron provistos por el proveedor (caso reservas de hotel puro), y genera los bytes del archivo PDF.

- reservacionID: identificador de la reservación a convertir en PDF
- []byte: bytes del PDF generado
- error: error si la reservación no existe, falla la serialización, el mapeo de datos o la generación del PDF

```
type PerfilService struct {
    PerfilService
```

Servicio encargado de gestionar las operaciones sobre el perfil del usuario, incluyendo la consulta de datos, actualización de teléfono y cambio de contraseña.

```
func NewPerfilService(db *sql.DB) *PerfilService
    NewPerfilService
```

Crea e inicializa una nueva instancia de PerfilService con su repositorio.

- *PerfilService: instancia inicializada del servicio de perfil

```
func (s *PerfilService) ActualizarTelefono(usuarioID int, telefono string) error
    ActualizarTelefono
```

Actualiza el numero de telefono del usuario identificado por su ID.

- usuarioID: identificador del usuario a actualizar
- telefono: nuevo numero de telefono a asignar
- error: error si falla la actualizacion en la base de datos

```
func (s *PerfilService) CambiarContraseña(usuarioID int, nueva string) error
    CambiarContraseña
```

Genera el hash bcrypt de la nueva contraseña y lo persiste en la base de datos para el usuario indicado.

- usuarioID: identificador del usuario que cambia su contraseña
- nueva: nueva contraseña en texto plano que sera hasheada antes de guardarse
- error: error si falla la generacion del hash o la actualizacion en la base de datos

```
func (s *PerfilService) ObtenerHash(usuarioID int) (string, error)
    ObtenerHash
```

Obtiene el hash de la contraseña actual del usuario para verificacion previa antes de permitir el cambio de contraseña.

- usuarioID: identificador del usuario cuyo hash se desea obtener
- string: hash bcrypt de la contraseña actual del usuario
- error: error si el usuario no existe o falla la consulta a la base de

```
func (s *PerfilService) ObtenerPerfil(usuarioID int) (map[string]interface{}, error)
    ObtenerPerfil
```

Recupera los datos del perfil del usuario identificado por su ID.

- usuarioID: identificador del usuario cuyo perfil se desea obtener
- map[string]interface{}: mapa con los campos del perfil del usuario

```
type ProveedorService struct {
    ProveedorService
```

Servicio encargado de la logica de negocio relacionada con la creacion y gestion de proveedores externos registrados en el sistema.

```
func NewProveedorService(db *sql.DB) *ProveedorService
```

```
NewProveedorService
```

Crea e inicializa una nueva instancia de ProveedorService con su repositorio.

- *ProveedorService: instancia inicializada del servicio de proveedores

```
func (s *ProveedorService) CrearProveedor(req dto.CrearProveedorRequest) (dto.CrearProveedorResponse, error)
```

```
CrearProveedor
```

Ejecuta las validaciones de negocio necesarias antes de registrar un nuevo proveedor: verifica que el usuario exista y tenga rol webservice (rol 3), que dicho usuario no tenga ya un proveedor asignado, y que el tipo de proveedor indicado sea valido. Si todas las validaciones pasan, delega la creacion al repositorio.

- req: datos del proveedor a crear, incluyendo el ID de usuario y tipo de
- dto.CrearProveedorResponse: datos del proveedor creado incluyendo token generado
- error: error si el usuario no existe, no tiene el rol correcto, ya tiene un proveedor asignado o el tipo de proveedor no existe

```
type ReservacionService struct {
```

```
ReservacionService
```

Servicio responsable de la creacion de nuevas reservaciones de viaje. Coordina la expiracion de reservaciones pendientes anteriores del usuario y la generacion de numeros de reservacion unicos con fecha de expiracion.

```
func NewReservacionService(db *sql.DB, expiracionService *ExpiracionService) *ReservacionService
```

```
NewReservacionService
```

Crea e inicializa una nueva instancia de ReservacionService con sus

- expiracionService: servicio que gestiona la expiracion de reservaciones pendientes
- *ReservacionService: instancia inicializada del servicio de reservaciones

```
func (s *ReservacionService) CrearReservacion(usuarioID, tipoReservaID int) (dto.CrearReservacionResponse, error)
    CrearReservacion
```

Crea una nueva reservacion para el usuario especificado. Primero expira cualquier reservacion pendiente anterior del mismo usuario, luego genera un numero de reservacion unico de 8 caracteres en mayusculas y calcula una fecha de expiracion de 10 minutos desde el momento de la creacion.

- usuarioID: identificador del usuario que crea la reservacion
- tipoReservaID: tipo de reserva (1=Aerolinea, 2=Hotel, 3=Paquete)
- dto.CrearReservacionResponse: datos de la reservacion creada incluyendo numero, estado y fechas
- error: error si falla la expiracion de reservaciones anteriores o la creacion en base de datos

```
type UbicacionResult struct {
    Pais    models.Pais    `json:"pais"`
    Ciudad  models.Ciudad  `json:"ciudad"`
    UbicacionResult
```

Estructura que agrupa el resultado de una busqueda o creacion de ubicacion, conteniendo el pais y la ciudad resueltos.

```
type UbicacionService struct {
    UbicacionService
```

Servicio encargado de resolver ubicaciones geograficas (países, ciudades y nacionalidades), creandolas en la base de datos si no existen previamente.

```
func NewUbicacionService(db *sql.DB) *UbicacionService
    NewUbicacionService
```

Crea e inicializa una nueva instancia de UbicacionService con su

- *UbicacionService: instancia inicializada del servicio de ubicaciones

```
func (s *UbicacionService) ObtenerOCrearNacionalidades(nombres []string) ([]models.Nacionalidad, error)
    ObtenerOCrearNacionalidades
```

Procesa un slice de nombres de nacionalidades, buscando cada una en la base de datos y creandola si no existe. Ignora entradas vacias o que solo contengan espacios. Recibe un slice porque el usuario puede tener multiples nacionalidades.

- nombres: slice de nombres de nacionalidades a resolver
- []models.Nacionalidad: slice con las nacionalidades resueltas
- error: error si falla la consulta o insercion de alguna nacionalidad en

```
func (s *UbicacionService) ObtenerOCrearUbicacion(nombreCiudad, nombrePais string) (UbicacionResult, error)
    ObtenerOCrearUbicacion
```

Busca en la base de datos el pais y la ciudad indicados por nombre. Si alguno no existe, lo crea automaticamente. Limpia espacios en blanco de los nombres antes de procesarlos.

- nombreCiudad: nombre de la ciudad a buscar o crear
- nombrePais: nombre del pais al que pertenece la ciudad
- UbicacionResult: struct con el pais y la ciudad resueltos
- error: error si falla la consulta o insercion en base de datos

```
type UsuarioService struct {
    UsuarioService
```

Servicio encargado de la logica de negocio para el registro y validacion de usuarios, incluyendo la resolucion de ubicaciones y nacionalidades mediante el servicio de ubicacion.

```
func NewUsuarioService(db *sql.DB, ubicacionService *UbicacionService) *UsuarioService
    NewUsuarioService
```

Crea e inicializa una nueva instancia de UsuarioService con sus

- ubicacionService: servicio de ubicaciones para resolver pais, ciudad y nacionalidades
- *UsuarioService: instancia inicializada del servicio de usuarios

```
func (s *UsuarioService) ObtenerTodos() ([]dto.UsuarioResumen, error)
    ObtenerTodos
```

Retorna la lista completa de usuarios registrados en el sistema con sus datos basicos y rol asignado. Usado por el panel de administracion para gestion de roles y asignacion de WebService.

- []dto.UsuarioResumen: lista de usuarios con id, nombre, apellido, correo y rol
- error: error si falla la consulta a la base de datos

```
func (s *UsuarioService) Registrar(req dto.RegistroUsuarioRequest) (dto.ValidacionUsuarioResponse, error)
    Registrar
```

Ejecuta el flujo completo de registro de un nuevo usuario: valida que correo, pasaporte y username no esten duplicados, resuelve la ubicacion geografica (pais y ciudad), resuelve las nacionalidades, crea el usuario en la base de datos con estado y rol por defecto (1), y le asigna sus nacionalidades. Si hay duplicados en los datos unicos, retorna la validacion sin registrar al usuario.

- req: datos completos del usuario a registrar
- dto.ValidacionUsuarioResponse: resultado de la validacion de datos unicos
- error: error si falla la validacion, la resolucion de ubicacion o la insercion en base de datos

```
func (s *UsuarioService) ValidarDatosUnicos(req dto.RegistroUsuarioRequest) (dto.ValidacionUsuarioResponse, error)
    ValidarDatosUnicos
```

Verifica si el correo, pasaporte o username del request ya existen en la base de datos. Devuelve un struct de validacion indicando cuales campos ya estan en uso.

- req: datos del usuario a registrar, incluyendo correo, pasaporte y username
- dto.ValidacionUsuarioResponse: flags indicando que campos ya existen en la base de datos
- error: error si falla alguna consulta a la base de datos

Repositories

Repositorios de acceso a datos para la agencia de viajes. Este paquete centraliza todas las consultas a la base de datos utilizadas por los servicios de la aplicacion.

TYPES

```
type AgenciaConfiguracionRepository struct {
```

```
    // Has unexported fields.
```

```
}
```

```
    AgenciaConfiguracionRepository
```

```
Repositorio que gestiona la lectura de la configuracion global de la
agencia, incluyendo el porcentaje de descuento aplicado a reservaciones de
tipo paquete (vuelo + hotel).
```

```
func NewAgenciaConfiguracionRepository(db *sql.DB) *AgenciaConfiguracionRepository
```

```
    NewAgenciaConfiguracionRepository
```

```
Crea e inicializa una nueva instancia de AgenciaConfiguracionRepository.
```

```
Parametros:
```

```
- db: conexion activa a la base de datos
```

```
Retorna:
```

```
- *AgenciaConfiguracionRepository: instancia lista para usar
```

```
func (r *AgenciaConfiguracionRepository) ObtenerPorcentajeDescuento() (float64, error)
```

```
    ObtenerPorcentajeDescuento
```

```
Retorna el porcentaje de descuento configurado para paquetes. Lee siempre el
primer registro de la tabla Agencia_Configuracion. Si la tabla esta vacia o
falla la consulta, retorna 0.
```

```
- float64: porcentaje de descuento (ej. 5.00 = 5%)
```

```
- error: error de base de datos, nil si la operacion fue exitosa
```

```
type BusquedaRepository struct {
```

```
    BusquedaRepository
```

```
Repositorio encargado de las consultas de busqueda de ciudades y proveedores
disponibles segun origen, destino y tipo de servicio.
```

```
func NewBusquedaRepository(db *sql.DB) *BusquedaRepository
    NewBusquedaRepository
```

Crea e inicializa una nueva instancia de BusquedaRepository.

- *BusquedaRepository: instancia lista para usar

```
func (r *BusquedaRepository) BuscarCiudadID(ciudad, pais string) (*int, error)
    BuscarCiudadID
```

Consulta el ID de una ciudad a partir de su nombre y el nombre del pais al que pertenece. La comparacion se realiza ignorando mayusculas y espacios.

- ciudad: nombre de la ciudad a buscar
- pais: nombre del pais al que pertenece la ciudad
- *int: puntero al ID de la ciudad encontrada, nil si no existe

```
func (r *BusquedaRepository) ObtenerAerolineasPorRuta(
    ciudadOrigenID, ciudadDestinoID int,
) ([]dto.ProveedorCatalogo, error)
    ObtenerAerolineasPorRuta
```

Consulta los proveedores de tipo aerolinea (Tipo_Catalogo_ID = 1) activos que operan desde una ciudad de origen determinada.

- ciudadOrigenID: ID de la ciudad de origen del vuelo
- ciudadDestinoID: ID de la ciudad de destino (recibido pero no usado en el filtro SQL)
- []dto.ProveedorCatalogo: lista de aerolineas disponibles para la ruta

Notas:

- El filtro por ciudad de destino no se aplica en la consulta actual

```
func (r *BusquedaRepository) ObtenerProveedoresPorOrigenYTipo(
    ciudadOrigenID, tipoCatalogoID int,
) []dto.ProveedorCatalogo
    ObtenerProveedoresPorOrigenYTipo
```

Consulta los proveedores activos que operan desde una ciudad de origen dada y que pertenecen a un tipo de catalogo especifico (por ejemplo, hotelero o aereo).

- ciudadOrigenID: ID de la ciudad de origen
- tipoCatalogoID: ID del tipo de catalogo (1=aerolinea, 2=hotelera)
- []dto.ProveedorCatalogo: lista de proveedores que cumplen el criterio

```
type CancelacionRepository struct {  
    CancelacionRepository
```

Repositorio que gestiona las operaciones de cancelacion de reservaciones, incluyendo la verificacion de pertenencia al usuario y la actualizacion del estado en base de datos mediante transacciones.

```
func NewCancelacionRepository(db *sql.DB) *CancelacionRepository  
    NewCancelacionRepository
```

Crea e inicializa una nueva instancia de CancelacionRepository.

- *CancelacionRepository: instancia lista para usar

```
func (r *CancelacionRepository) CancelarReservacion(reservacionID int, motivo string) error  
    CancelarReservacion
```

Marca la reservacion y todos sus detalles activos como cancelados (estado 3) dentro de una transaccion atomica. Ademas registra la fecha de cancelacion y el motivo proporcionado.

- reservacionID: ID de la reservacion a cancelar
- motivo: descripcion del motivo de cancelacion
- error: error si alguna operacion de la transaccion falla, nil si fue exitosa
- Si cualquier paso falla, se realiza rollback automatico

```
func (r *CancelacionRepository) ObtenerDetallesParaCancelar(reservacionID int) ([]dto.DetalleProveedor, error)  
    ObtenerDetallesParaCancelar
```

Recupera los detalles de una reservacion junto con los datos del proveedor asociado a cada detalle. Solo incluye detalles con estado pendiente (1) o confirmado (2).

- reservacionID: ID de la reservacion cuyos detalles se desean obtener
- []dto.DetalleProveedor: lista de detalles con informacion del proveedor

```
func (r *CancelacionRepository) ObtenerReservacionParaCancelar(reservacionID, usuarioID int) (estadoID int, err error)
```

ObtenerReservacionParaCancelar

Verifica que una reservacion exista, pertenezca al usuario indicado y retorna su estado actual para validar si es cancelable.

- reservacionID: ID de la reservacion a consultar
- usuarioID: ID del usuario que solicita la cancelacion
- estadoID: identificador del estado actual de la reservacion
- error: error si la reservacion no existe, no pertenece al usuario o falla la consulta

```
type CatalogoRepository struct {
```

CatalogoRepository

Repositorio encargado de las operaciones sobre el catalogo de proveedores, incluyendo consulta de tipos, datos de conexion, sincronizacion y actualizacion de rutas disponibles por proveedor.

```
func NewCatalogoRepository(db *sql.DB) *CatalogoRepository
```

NewCatalogoRepository

Crea e inicializa una nueva instancia de CatalogoRepository.

- *CatalogoRepository: instancia lista para usar

```
func (r *CatalogoRepository) EliminarCatalogoPorProveedor(proveedorID int) error
```

EliminarCatalogoPorProveedor

Elimina todas las entradas del catalogo asociadas a un proveedor especifico. Utilizado antes de sincronizar el catalogo con datos actualizados del webservice.

- proveedorID: ID del proveedor cuyo catalogo se desea eliminar

```
func (r *CatalogoRepository) InsertarEntrada(
    ciudadOrigenID int,
    ciudadDestinoID *int,
    tipoCatalogoID int,
    proveedorID int,
```

```
) error
```

```
    InsertarEntrada
```

Registra una nueva entrada en el catalogo del proveedor, asociando una ciudad de origen, opcionalmente una ciudad de destino, el tipo de catalogo y el proveedor.

- ciudadOrigenID: ID de la ciudad de origen del servicio
- ciudadDestinoID: puntero al ID de la ciudad de destino; puede ser nil para servicios hoteleros
- proveedorID: ID del proveedor al que pertenece esta entrada
- Si ciudadDestinoID es nil, se inserta NULL en la columna Ciudad_Destino_ID

```
func (r *CatalogoRepository) ObtenerDatosConexion(proveedorID int) (urlAPI, tokenEntrada string, err error)
    ObtenerDatosConexion
```

Recupera la URL de la API y el token de entrada del proveedor especificado, necesarios para realizar llamadas al webservice externo.

- proveedorID: ID del proveedor a consultar
- urlAPI: URL base del API del proveedor
- tokenEntrada: token de autenticacion para las peticiones al proveedor

```
func (r *CatalogoRepository) ObtenerProveedoresActivos() ([]int, error)
    ObtenerProveedoresActivos
```

Recupera los IDs de todos los proveedores cuyo estado es activo (EstadoID = 1).

- (ninguno)
- []int: lista de IDs de proveedores activos

```
func (r *CatalogoRepository) ObtenerTipoProveedor(proveedorID int) (int, error)
    ObtenerTipoProveedor
```

Consulta el tipo de proveedor asociado a un ID dado. El tipo determina la categoria del servicio (1=aerolinea, 2=hotelera).

- int: ID del tipo de proveedor

```
type DetalleReservacionRepository struct {
    DetalleReservacionRepository
```

Repositorio que gestiona las operaciones sobre los detalles de reservacion, incluyendo la insercion de nuevos detalles, consulta de datos de proveedor, actualizacion de totales y validacion de vuelos pendientes.

```
func NewDetalleReservacionRepository(db *sql.DB) *DetalleReservacionRepository
    NewDetalleReservacionRepository
```

Crea e inicializa una nueva instancia de DetalleReservacionRepository.

- *DetalleReservacionRepository: instancia lista para usar

```
func (r *DetalleReservacionRepository) ActualizarTotalReservacion(reservacionID int, montoAgregar float64)
    error
```

ActualizarTotalReservacion

Incrementa el total acumulado de una reservacion sumando el monto indicado. Se utiliza cada vez que se agrega un nuevo detalle a la reservacion.

- reservacionID: ID de la reservacion a actualizar
- montoAgregar: monto que se suma al total existente

```
func (r *DetalleReservacionRepository) InsertarDetalle(
    reservacionID, proveedorID, tipoDetalleID int,
    idReservaProveedor string,
    total float64,
    parametrosJson interface{},
    InsertarDetalle
```

Inserta un nuevo registro en la tabla Detalles_Reservacion con estado pendiente (1). Los parametros adicionales de la reserva se serializan a JSON antes de persistirse.

- reservacionID: ID de la reservacion padre
- proveedorID: ID del proveedor que gestiona este detalle
- tipoDetalleID: tipo de servicio reservado (1=vuelo, 2=hotel)
- idReservaProveedor: identificador de la reserva en el sistema del proveedor externo
- total: monto total del detalle con el margen de ganancia aplicado
- parametrosJson: estructura con los parametros de la reserva a serializar
- error: error de serializacion o base de datos, nil si la operacion fue

```
func (r *DetalleReservacionRepository) ObtenerDatosProveedor(proveedorID int) (urlAPI, tokenEntrada string, porcentajeGanancia float64, err error)
```

ObtenerDatosProveedor

Recupera la URL del API, el token de entrada y el porcentaje de ganancia configurado para un proveedor específico.

- tokenEntrada: token de autenticación para peticiones al proveedor
- porcentajeGanancia: margen de ganancia configurado para el proveedor

```
func (r *DetalleReservacionRepository) ObtenerDatosProveedorPorTipo(
    proveedorID, tipoDetalleID int,
) (urlAPI, tokenEntrada string, porcentajeGanancia float64, err error)
ObtenerDatosProveedorPorTipo
```

Recupera los datos de conexión de un proveedor validando además que coincida con el tipo de detalle solicitado (1=aerolínea, 2=hotelera).

- tipoDetalleID: ID del tipo de proveedor esperado
- porcentajeGanancia: margen de ganancia configurado
- error: error si el proveedor no existe o no corresponde al tipo, nil si fue exitosa

```
func (r *DetalleReservacionRepository) ObtenerDetalleAerolineaPorProveedor(
    reservacionID, usuarioID, proveedorID int,
) (idReservaProveedor string, urlAPI string, tokenEntrada string, err error)
ObtenerDetalleAerolineaPorProveedor
```

Busca el identificador de reserva en el proveedor aerolínea para un detalle de tipo vuelo (Tipo_Detalle_ID = 1) que este pendiente, validando que la reservación pertenezca al usuario y este en estado activo.

- usuarioID: ID del usuario propietario de la reservación
 - proveedorID: ID del proveedor aerolínea involucrado
- proveedor
- tokenEntrada: token de autenticación del proveedor
 - error: error si no se encuentra el detalle o la reservación no pertenece al usuario

```
func (r *DetalleReservacionRepository) ObtenerReservacionParaDetalle(reservacionID, usuarioID int) (*dto.ReservacionValidada, error) {
    ObtenerReservacionParaDetalle
```

Verifica que una reservacion exista y pertenezca al usuario indicado, retornando sus datos basicos para validacion previa a la insercion de detalles.

- *dto.ReservacionValidada: datos basicos de la reservacion, nil si no existe

```
func (r *DetalleReservacionRepository) RecalcularTotalReservacion(reservacionID int) error {
    RecalcularTotalReservacion
```

Recalcula y actualiza el total de una reservacion sumando el total de todos sus detalles que se encuentren en estado pendiente (Estado_Detalle_ID = 1).

- reservacionID: ID de la reservacion a recalcular
- Utiliza COALESCE para retornar 0 si no hay detalles activos

```
type LoginRepository struct {
    LoginRepository
```

Repository encargado de las consultas necesarias para el proceso de autenticacion de usuarios, permitiendo buscar por nombre de usuario o por correo electronico.

```
func NewLoginRepository(db *sql.DB) *LoginRepository {
    NewLoginRepository
```

Crea e inicializa una nueva instancia de LoginRepository.

- *LoginRepository: instancia lista para usar

```
func (r *LoginRepository) ObtenerPorUsernameOCorreo(login string) (models.Usuario, error) {
    ObtenerPorUsernameOCorreo
```

Busca un usuario en la base de datos comparando el valor recibido contra el campo Username y el campo Correo. Retorna el modelo completo del usuario si se encuentra una coincidencia, o un modelo vacio si no existe.

- login: valor a buscar, puede ser el username o el correo del usuario
- models.Usuario: datos completos del usuario encontrado, vacio si no
- Si no se encuentra el usuario se retorna un struct vacio sin error

```
type MisReservacionesRepository struct {
    MisReservacionesRepository
```

Repositorio que gestiona la consulta del historial de reservaciones de un usuario, retornando tanto los datos de la reservacion como sus detalles y la informacion del proveedor asociado a cada uno.

```
func NewMisReservacionesRepository(db *sql.DB) *MisReservacionesRepository
    NewMisReservacionesRepository
```

Crea e inicializa una nueva instancia de MisReservacionesRepository.

- *MisReservacionesRepository: instancia lista para usar

```
func (r *MisReservacionesRepository) ObtenerReservacionPorID(reservacionID, usuarioID int)
([]dto.FilaReservacionDetalle, error)
    ObtenerReservacionPorID
```

Recupera una reservacion especifica junto con todos sus detalles y los datos del proveedor, verificando que la reservacion pertenezca al usuario indicado.

- []dto.FilaReservacionDetalle: filas planas con datos de reservacion, detalle y proveedor

```
func (r *MisReservacionesRepository) ObtenerReservacionesDeUsuario(usuarioID int) ([]dto.FilaReservacionDetalle,
error)
    ObtenerReservacionesDeUsuario
```

Recupera todas las reservaciones de un usuario junto con sus detalles y los datos del proveedor de cada detalle. Los resultados se ordenan por fecha de creacion descendente.

- usuarioID: ID del usuario cuyas reservaciones se desean consultar

```
type PagoRepository struct {
    PagoRepository
```

Repositorio encargado de las operaciones relacionadas con el proceso de pago, incluyendo la validacion de la reserva, el conteo de detalles por tipo y la confirmacion atomica de la reserva junto con la creacion de la factura.

```
func NewPagoRepository(db *sql.DB) *PagoRepository
    NewPagoRepository
```

Crea e inicializa una nueva instancia de PagoRepository.

- *PagoRepository: instancia lista para usar

```
func (r *PagoRepository) ConfirmarReservaYFacturar(reservacionID int, total float64, nit string, codigoPostal
string, porcentajeDescuento float64) error
    ConfirmarReservaYFacturar
```

Ejecuta dentro de una transaccion atomica los pasos del proceso de confirmacion: aplica el descuento de paquete si corresponde, actualiza el total de la reservacion, cambia el estado a confirmada (2), confirma todos sus detalles y crea la factura. El descuento es absorbido por la agencia y se aplica unicamente a reservaciones de tipo paquete (Tipo_Reserva_ID = 3).

- reservacionID: ID de la reservacion a confirmar
- total: monto total original de la reservacion antes del descuento
- nit: numero de identificacion tributaria del cliente para la factura
- codigoPostal: codigo postal del cliente para la factura
- porcentajeDescuento: porcentaje de descuento a aplicar (0 si no aplica)
- Si cualquier paso falla, se realiza rollback automatico de toda la transaccion
- El descuento se redondea a 2 decimales antes de aplicarse

```
func (r *PagoRepository) ContarDetallesPorTipo(reservacionID int) (vuelos int, hoteles int, err error)
    ContarDetallesPorTipo
```

Cuenta cuantos detalles de tipo vuelo (1) y tipo hotel (2) tiene una reservacion, considerando unicamente los detalles en estado pendiente (Estado_Detalle_ID = 1).

- reservacionID: ID de la reservacion a evaluar
- vuelos: cantidad de detalles de tipo vuelo pendientes
- hoteles: cantidad de detalles de tipo hotel pendientes

```
func (r *PagoRepository) ObtenerReservaParaPago(reservacionID, usuarioID int) (tipoReserva int, total float64, err error)
    ObtenerReservaParaPago
```

Verifica que la reserva pertenezca al usuario indicado y se encuentre en estado pendiente (EstadoID = 1), retornando el tipo de reserva y el total.

- tipoReserva: ID del tipo de reserva (vuelo, hotel, paquete, etc.)
- total: monto total acumulado de la reservacion
- error: error si la reserva no existe, no pertenece al usuario o no esta pendiente

```
type PerfilRepository struct {
    PerfilRepository
```

Repository encargado de las operaciones de consulta y actualizacion del perfil del usuario autenticado, incluyendo datos personales, ubicacion, nacionalidades y gestion de contrasena.

```
func NewPerfilRepository(db *sql.DB) *PerfilRepository
    NewPerfilRepository
```

Crea e inicializa una nueva instancia de PerfilRepository.

- *PerfilRepository: instancia lista para usar

```
func (r *PerfilRepository) ActualizarContrasena(usuarioID int, hash string) error
    ActualizarContrasena
```

Actualiza el hash de la contrasena del usuario en la base de datos. Debe recibir el hash ya procesado, no la contrasena en texto plano.

- usuarioID: ID del usuario cuya contrasena se desea actualizar
- hash: nuevo hash bcrypt de la contrasena a persistir

```
func (r *PerfilRepository) ActualizarTelefono(usuarioID int, telefono string) error
    ActualizarTelefono
```

Actualiza el numero de telefono del usuario identificado por su ID.

- usuarioID: ID del usuario a actualizar
- telefono: nuevo numero de telefono a registrar

```
func (r *PerfilRepository) ObtenerHash(usuarioID int) (string, error)
```

ObtenerHash

Recupera el hash de la contraseña actual del usuario, utilizado para validar la contraseña anterior antes de permitir un cambio.

- usuarioID: ID del usuario cuyo hash se desea obtener
- string: hash bcrypt de la contraseña actual

```
func (r *PerfilRepository) ObtenerPerfil(usuarioID int) (map[string]interface{}, error)
```

ObtenerPerfil

Recupera todos los datos del perfil del usuario incluyendo nombre, apellido, correo, username, pasaporte, telefono, fecha de nacimiento, ciudad, pais y la lista de nacionalidades asociadas.

- usuarioID: ID del usuario cuyo perfil se desea consultar
- map[string]interface{}: mapa con todos los campos del perfil del usuario
- Ciudad y pais pueden ser NULL si el usuario no tiene ciudad asignada
- La lista de nacionalidades puede estar vacia si no se asignaron al registrarse

```
type ProveedorRepository struct {
```

ProveedorRepository

Repositorio encargado de las operaciones sobre la entidad Proveedor, incluyendo creacion, validacion de roles y existencia, almacenamiento de tokens y consulta de datos de conexion por distintos criterios.

```
func NewProveedorRepository(db *sql.DB) *ProveedorRepository
```

NewProveedorRepository

Crea e inicializa una nueva instancia de ProveedorRepository.

- *ProveedorRepository: instancia lista para usar

```
func (r *ProveedorRepository) CrearProveedor(req dto.CrearProveedorRequest) (dto.CrearProveedorResponse, error)  
    CrearProveedor
```

Inserta un nuevo proveedor en la base de datos con estado activo (EstadoID = 1) y tokens vacios que seran generados y guardados posteriormente.

- req: DTO con los datos necesarios para crear el proveedor
- dto.CrearProveedorResponse: datos del proveedor recién creado incluyendo su ID

```
func (r *ProveedorRepository) ExisteTipoProveedor(tipoID int) (bool, error)  
    ExisteTipoProveedor
```

Verifica si existe un registro en la tabla Tipo_Proveedor con el ID

- tipoID: ID del tipo de proveedor a validar
- bool: true si el tipo existe, false en caso contrario

```
func (r *ProveedorRepository) GuardarTokens(proveedorID int, tokenEntrada, tokenSalida string) error  
    GuardarTokens
```

Actualiza los tokens de entrada y salida de un proveedor existente.
Se utiliza luego de la creacion del proveedor para persistir los hashes generados.

- proveedorID: ID del proveedor al que se asignan los tokens
- tokenEntrada: hash del token de entrada para autenticar peticiones entrantes
- tokenSalida: hash del token de salida para autenticar peticiones salientes

```
func (r *ProveedorRepository) ObtenerProveedorPorID(proveedorID int) (*dto.DetalleProveedor, error)  
    ObtenerProveedorPorID
```

Recupera la URL del API y el token de entrada de un proveedor específico verificando que se encuentre en estado activo (EstadoID = 1).

- *dto.DetalleProveedor: datos de conexión del proveedor encontrado
- error: error si el proveedor no existe o está inactivo, nil si fue

```
func (r *ProveedorRepository) ObtenerProveedorPorTipo(tipoProveedorID int) (*dto.DetalleProveedor, error)
    ObtenerProveedorPorTipo
```

Recupera la URL del API y el token de entrada del primer proveedor activo que corresponda al tipo indicado. Retorna error si no hay proveedores activos de ese tipo.

- tipoProveedorID: ID del tipo de proveedor a buscar (1=aerolinea, 2=hotelera)
- error: error si no hay proveedor activo del tipo indicado o falla la consulta

```
func (r *ProveedorRepository) ObtenerProveedorPorToken(token string) (*dto.ProveedorIdentidad, error)
    ObtenerProveedorPorToken
```

Busca un proveedor utilizando su token de entrada (Token_HASH_Entrada). Retorna nil si no se encuentra ningún proveedor con ese token.

- token: hash del token de entrada a buscar
- *dto.ProveedorIdentidad: datos de identidad del proveedor encontrado, nil si no existe

```
func (r *ProveedorRepository) ObtenerRolUsuario(usuarioID int) (int, error)
    ObtenerRolUsuario
```

Consulta el RolID asignado a un usuario específico. Retorna 0 si el usuario no existe en la base de datos.

- usuarioID: ID del usuario a consultar
- int: ID del rol del usuario, 0 si no existe

```
func (r *ProveedorRepository) ObtenerURLAPI(proveedorID int) (string, error)
    ObtenerURLAPI
```

Recupera la URL del API de un proveedor específico para realizar llamadas al webservice externo.

- string: URL del API del proveedor

```
func (r *ProveedorRepository) UsuarioYaTieneProveedor(usuarioID int) (bool, error)
    UsuarioYaTieneProveedor
```

Verifica si un usuario webservice ya tiene un proveedor registrado y asociado a su cuenta, para evitar duplicidad.

- usuarioID: ID del usuario a verificar
- bool: true si ya existe un proveedor asociado, false en caso contrario

```
type ReservacionRepository struct {
    ReservacionRepository
```

Repositorio que gestiona el ciclo de vida de las reservaciones en la base de datos, incluyendo creacion, consulta de pendientes, expiracion masiva e individual, y recuperacion de detalles asociados a cada reservacion.

```
func NewReservacionRepository(db *sql.DB) *ReservacionRepository
    NewReservacionRepository
```

Crea e inicializa una nueva instancia de ReservacionRepository.

- *ReservacionRepository: instancia lista para usar

```
func (r *ReservacionRepository) CrearReservacion(
    usuarioID int,
    tipoReservaID int,
    noReservacion string,
    fechaExpiracion string,
) (int, error)
    CrearReservacion
```

Inserta una nueva reservacion en la base de datos con estado pendiente (1) y total inicial de cero.

- usuarioID: ID del usuario que realiza la reservacion
- tipoReservaID: tipo de reserva (vuelo, hotel, paquete, etc.)
- noReservacion: numero unico de reservacion generado por el servicio
- fechaExpiracion: fecha y hora limite para confirmar la reservacion
- int: ID autogenerado de la nueva reservacion

```
func (r *ReservacionRepository) ExpirarDetalles(reservacionID int) error
    ExpirarDetalles
```

Marca como cancelados (estado 3) todos los detalles de una reservacion, utilizado durante el proceso de expiracion de la reservacion padre.

- reservacionID: ID de la reservacion cuyos detalles se deben cancelar

```
func (r *ReservacionRepository) ExpirarReservacion(reservacionID int) error
    ExpirarReservacion
```

Marca una reservacion especifica como expirada (estado 4) siempre que actualmente se encuentre en estado pendiente (1).

- reservacionID: ID de la reservacion a expirar

```
func (r *ReservacionRepository) ExpirarReservacionesPendientes() error
    ExpirarReservacionesPendientes
```

Actualiza a estado expirado (4) todas las reservaciones que se encuentren en estado pendiente (1) y cuya fecha de expiracion ya haya pasado.

```
func (r *ReservacionRepository) ObtenerDetallesDeReservacion(reservacionID int) ([]dto.DetalleProveedor, error)
    ObtenerDetallesDeReservacion
```

Recupera todos los detalles de una reservacion junto con los datos de conexion del proveedor asociado a cada detalle.

```
func (r *ReservacionRepository) ObtenerIDsPendientesExpirados() ([]int, error)
    ObtenerIDsPendientesExpirados
```

Consulta los IDs de todas las reservaciones que esten en estado pendiente (1) y cuya fecha de expiracion ya haya pasado. Usado por el scheduler de expiracion.

- []int: lista de IDs de reservaciones pendientes expiradas

```
func (r *ReservacionRepository) ObtenerPendientesConDetalles(usuarioID int) ([]dto.ReservacionConDetalles, error)
    ObtenerPendientesConDetalles
```

Recupera todas las reservaciones pendientes de un usuario junto con los detalles y datos del proveedor asociados a cada una.

- usuarioID: ID del usuario cuyas reservaciones pendientes se desean consultar
- []dto.ReservacionConDetalles: lista de reservaciones con sus detalles de

```
type UbicacionRepository struct {
    UbicacionRepository
```

Repositorio encargado de la gestion de entidades geograficas como paises, ciudades y nacionalidades. Implementa el patron buscar-o-crear para garantizar que no se dupliquen registros existentes en la base de datos.

```
func NewUbicacionRepository(db *sql.DB) *UbicacionRepository
    NewUbicacionRepository
```

Crea e inicializa una nueva instancia de UbicacionRepository.

- *UbicacionRepository: instancia lista para usar

```
func (r *UbicacionRepository) BuscarOCrearCiudad(nombre string, paisID int) (models.Ciudad, error)
    BuscarOCrearCiudad
```

Busca una ciudad por nombre y pais en la base de datos. Si no existe, la inserta y retorna el registro recién creado con su ID autogenerado.

- nombre: nombre de la ciudad a buscar o crear
- paisID: ID del pais al que pertenece la ciudad
- models.Ciudad: entidad de la ciudad encontrada o creada

```
func (r *UbicacionRepository) BuscarOCrearNacionalidad(nombre string) (models.Nacionalidad, error)
    BuscarOCrearNacionalidad
```

Busca una nacionalidad por nombre en la base de datos. Si no existe,

- nombre: nombre de la nacionalidad a buscar o crear
- models.Nacionalidad: entidad de la nacionalidad encontrada o creada

```
func (r *UbicacionRepository) BuscarOCrearPais(nombre string) (models.Pais, error)
```

```
    BuscarOCrearPais
```

Busca un pais por nombre en la base de datos. Si no existe, lo inserta y retorna el registro recién creado con su ID autogenerado.

- nombre: nombre del pais a buscar o crear
- models.Pais: entidad del pais encontrado o creado

```
type UsuarioRepository struct {
```

```
    UsuarioRepository
```

Repositorio encargado de las operaciones sobre la entidad Usuario, incluyendo validacion de unicidad de campos, creacion de nuevos usuarios, asignacion de nacionalidades y consulta de datos de contacto.

```
func NewUsuarioRepository(db *sql.DB) *UsuarioRepository
```

```
    NewUsuarioRepository
```

Crea e inicializa una nueva instancia de UsuarioRepository.

- *UsuarioRepository: instancia lista para usar

```
func (r *UsuarioRepository) AsignarNacionalidades(usuarioID int, nacionalidadIDs []int) error
```

```
    AsignarNacionalidades
```

Inserta las asociaciones entre un usuario y sus nacionalidades en la tabla UsuarioNacionalidad. Itera sobre la lista de IDs de nacionalidad proporcionada.

- usuarioID: ID del usuario al que se le asignan las nacionalidades
- nacionalidadIDs: lista de IDs de nacionalidades a asociar
- error: error de base de datos si alguna insercion falla, nil si todas fueron exitosas

```
func (r *UsuarioRepository) CrearUsuario(req dto.RegistroUsuarioRequest, ciudadID, rolID, estadoID int) (int, error)
    CrearUsuario
```

Inserta un nuevo usuario en la base de datos. La contraseña se hashea antes de persistirse utilizando el helper de seguridad.

- req: DTO con los datos del formulario de registro del usuario
- ciudadID: ID de la ciudad de residencia del usuario
- rolID: ID del rol asignado al nuevo usuario
- estadoID: ID del estado inicial del usuario (activo, pendiente, etc.)
- int: ID autogenerado del usuario recién creado
- error: error de hasheo o de base de datos, nil si la operación fue

```
func (r *UsuarioRepository) ExisteCorreo(correo string) (bool, error)
    ExisteCorreo
```

Verifica si ya existe un usuario registrado con el correo electrónico

- correo: correo electrónico a verificar
- bool: true si el correo ya está en uso, false en caso contrario

```
func (r *UsuarioRepository) ExistePasaporte(pasaporte string) (bool, error)
    ExistePasaporte
```

Verifica si ya existe un usuario registrado con el número de pasaporte

- pasaporte: número de pasaporte a verificar
- bool: true si el pasaporte ya está en uso, false en caso contrario

```
func (r *UsuarioRepository) ExisteUsername(username string) (bool, error)
    ExisteUsername
```

Verifica si ya existe un usuario registrado con el nombre de usuario

- username: nombre de usuario a verificar
- bool: true si el username ya está en uso, false en caso contrario

```
func (r *UsuarioRepository) ObtenerNombreYEmail(usuarioID int) (nombre, email string, err error)
    ObtenerNombreYEmail
```

Recupera el nombre completo y el correo electronico de un usuario por su ID.
El nombre completo se construye concatenando Nombre y Apellido en Go para evitar problemas de concatenacion en distintos motores de base de datos.

- nombre: nombre completo del usuario (Nombre + Apellido)
- email: correo electronico del usuario

```
func (r *UsuarioRepository) ObtenerTodos() ([]dto.UsuarioResumen, error)
    ObtenerTodos
```

Recupera la lista completa de usuarios registrados en el sistema, incluyendo su rol asignado mediante un JOIN con la tabla rol. Usado por el panel de administracion para gestion de roles y asignacion de usuarios WebService a proveedores.

- []dto.UsuarioResumen: lista de usuarios con id, nombre, apellido, correo y rol

ESTRUCTURA

Models

Define las estructuras de dominio utilizadas en toda la aplicacion Movent para representar las entidades de la base de datos.

TYPES

```
type Ciudad struct {
    ID      int    `json:"id"`
    Nombre  string  `json:"nombre"`
    PaisID  int    `json:"pais_id"`
}
```

Ciudad

Representa una ciudad registrada en el sistema. Se utiliza para asociar usuarios y ubicaciones geograficas dentro de la plataforma de reservaciones.

```
type Nacionalidad struct {
    Nacionalidad
```

Representa una nacionalidad disponible en el sistema. Se asocia a los usuarios durante el registro para registrar su origen o ciudadania.

```
type Pais struct {
    Pais
```

Representa un pais registrado en el sistema. Se utiliza como referencia geografica para ciudades, usuarios y destinos de viaje.

```
type Usuario struct {  
    ID            int    `json:"id"`  
    Nombre        string `json:"nombre"`  
    Apellido      string `json:"apellido"`  
    Correo        string `json:"correo"`  
    Username      string `json:"username"`  
    Contraseña    string `json:"-"`  
    Pasaporte     string `json:"pasaporte"`  
    Telefono      string `json:"telefono"`  
    FechaNacimiento string `json:"fecha_nacimiento"`  
    CiudadID      int    `json:"ciudad_id"`  
    RolID         int    `json:"rol_id"`  
    EstadoID      int    `json:"estado_id"`  
}
```

Usuario

Representa un usuario registrado en la plataforma Movent. Contiene datos personales, credenciales de acceso y referencias a su ciudad, rol y estado dentro del sistema.

Notas:

- El campo Contraseña usa la etiqueta `json:"-"` para evitar que sea serializado en las respuestas HTTP

DTO

Contiene los Data Transfer Objects utilizados para la comunicacion entre la capa de transporte y la capa de servicio de la agencia de viajes.

TYPES

```
type ActualizarCatalogoResponse struct {
    Proveedor string `json:"proveedor"` // Nombre del proveedor cuyo catalogo fue actualizado
    Insertados int  `json:"insertados"` // Cantidad de registros nuevos insertados en la base de datos
    Mensaje    string `json:"mensaje"` // Mensaje descriptivo del resultado de la operacion
}
```

ActualizarCatalogoResponse

Representa la respuesta retornada al cliente tras ejecutar la actualizacion del catalogo de rutas o habitaciones de un proveedor.

```
type AgregarDetalleHotelRequest struct {
    ReservacionID int          `json:"reservacionId"` // ID de la reservacion a la que se agrega el
detalle
    ProveedorID    int          `json:"proveedorId"` // ID del proveedor hotelero
    Habitaciones   []SeleccionHabitacion `json:"habitaciones"` // Lista de habitaciones seleccionadas con fechas y
personas
}
```

AgregarDetalleHotelRequest

Representa la solicitud para agregar una o varias habitaciones seleccionadas como detalle de hotel dentro de una reservacion existente.

```
type AgregarDetalleVueloRequest struct {
    ReservacionID int          `json:"reservacion_id"` // ID de la reservacion a la que se agrega el
detalle
    ProveedorID    int          `json:"proveedor_id"` // ID del proveedor aerolinea
    Vuelos         []SeleccionVuelo `json:"vuelos"` // Lista de vuelos seleccionados con clase y cantidad de
pasajeros
}
```

AgregarDetalleVueloRequest

Representa la solicitud para agregar uno o varios vuelos seleccionados como detalle dentro de una reservacion existente.

```

type AgregarPasajerosVueloAerolineaBody struct {
    ReservacionID int          `json:"reservacionId"` // ID de la reservacion en el sistema de la
aerolinea
    Pasajeros      []PasajeroVueloDTO `json:"pasajeros"`    // Lista de pasajeros con sus datos personales
    AgregarPasajerosVueloAerolineaBody

```

Representa el cuerpo de la solicitud que se envía directamente a la API de la aerolínea para registrar los datos de pasajeros en la reservación del proveedor.

```

type AgregarPasajerosVueloRequest struct {
    ReservacionID int          `json:"reservacion_id"` // ID de la reservacion en la base de datos de la
agencia
    ProveedorID   int          `json:"proveedor_id"`   // ID del proveedor aerolinea
    Pasajeros      []PasajeroVueloDTO `json:"pasajeros"`    // Lista de pasajeros con sus datos personales
    AgregarPasajerosVueloRequest

```

Representa la solicitud recibida por el endpoint de la agencia para agregar los datos de pasajeros a los boletos de un vuelo reservado.

```

type AsientoVueloDetalleDTO struct {
    VueloID          int          `json:"vueloId"`        // ID unico del vuelo
    NumeroVuelo      string       `json:"numeroVuelo"`    // Numero identificador del vuelo
    CapacidadPasajeros int          `json:"capacidadPasajeros"` // Capacidad total de pasajeros
    Columnas         []string     `json:"columnas"`       // Letras de columnas disponibles (ej.
["A","B","C"])
    FilasEjecutiva   int          `json:"filasEjecutiva"` // Numero de filas de clase ejecutiva
    TotalFilas       int          `json:"totalFilas"`     // Total de filas en el avion
    AsientosOcupados []string     `json:"asientosOcupados"` // Lista de codigos de asientos ya ocupados
    BoletosAgencia   []BoletoAsientoDTO `json:"boletosAgencia"` // Boletos pertenecientes a la agencia en
este vuelo
    AsientoVueloDetalleDTO

```

Representa la información detallada del mapa de asientos de un vuelo individual, incluyendo ocupación y boletos de la agencia.

```

type AsientosVueloResponse []AsientoVueloDetalleDTO
AsientosVueloResponse

```

Representa la respuesta completa de consulta de asientos, siendo una lista de detalles por cada vuelo de la reservación.

```

type BoletoAsientoDTO struct {
    BoletoID int    `json:"boletoId"` // ID unico del boleto
    NoBoleto string `json:"noBoleto"` // Numero de boleto legible
    Asiento  string `json:"asiento"`   //Codigo de asiento asignado (ej. "12A")
    ClaseID  int    `json:"claseId"`   // ID de la clase de servicio
    Clase    string `json:"clase"`     // Nombre de la clase de servicio (ej. "Economica")
}
BoletoAsientoDTO

```

Representa los datos de un boleto individual con su asiento y clase asignados dentro de un vuelo.

```

type BusquedaHotelesRequest struct {
    Ciudad      string `json:"ciudad"`      // Ciudad donde se busca el hotel
    Pais        string `json:"pais"`        // Pais donde se busca el hotel
    FechaCheckIn string `json:"fechaCheckIn"` // Fecha de entrada en formato ISO 8601
    FechaCheckOut string `json:"fechaCheckOut"` // Fecha de salida en formato ISO 8601
    CantidadPersonas int `json:"cantidadPersonas"` // Numero de personas que se hospedan
}
BusquedaHotelesRequest

```

Representa los criterios de búsqueda enviados por el cliente para consultar hoteles disponibles en todos los proveedores hoteleros. Es compartido por las rutas de búsqueda individual y general.

```

type BusquedaHotelesResponse struct {
    Proveedor  string    `json:"proveedor"` // Nombre del proveedor hotelero
    ProveedorID int       `json:"proveedor_id"` // ID del proveedor hotelero
    Datos      interface{} `json:"datos"`      // Datos crudos de hoteles devueltos por el proveedor
    Error      string    `json:"error,omitempty"` // Mensaje de error si la consulta al proveedor fallo
}
BusquedaHotelesResponse

```

Representa la respuesta de búsqueda de hoteles para un proveedor específico, encapsulando los datos crudos devueltos por su API o el error ocurrido.

```

type BusquedaVuelosRequest struct {
    Origen      string `json:"origen"`      // Ciudad de origen del vuelo
    OrigenPais  string `json:"origenPais"`  // Pais de origen del vuelo
    Destino     string `json:"destino"`     // Ciudad de destino del vuelo
    DestinoPais string `json:"destinoPais"` // Pais de destino del vuelo
    Fecha       string `json:"fecha"`       // Fecha del vuelo en formato ISO 8601
    CantidadPasajeros int `json:"cantidadPasajeros"` // Numero de pasajeros requeridos
}
BusquedaVuelosRequest

```

vuelos disponibles en todos los proveedores aerolínea. Es compartido por las rutas de búsqueda individual y general.

```

type BusquedaVuelosResponse struct {
    Proveedor    string    `json:"proveedor"`    // Nombre del proveedor aerolinea
    ProveedorID  int        `json:"proveedor_id"` // ID del proveedor aerolinea
    Datos        interface{} `json:"datos"`        // Datos crudos de vuelos devueltos por el proveedor
    BusquedaVuelosResponse

```

Representa la respuesta de búsqueda de vuelos para un proveedor específico,

```

type CambiarAsientoAerolineaBody struct {
    NuevoAsiento string `json:"nuevoAsiento"` // Código del nuevo asiento a asignar
    CambiarAsientoAerolineaBody

```

la aerolínea para actualizar el asiento de un pasajero.

```

type CambiarAsientoVueloRequest struct {
    ReservacionID int    `json:"reservacion_id"` // ID de la reservación en la base de datos de la agencia
    ProveedorID   int    `json:"proveedor_id"`   // ID del proveedor aerolínea
    BoletoID      int    `json:"boleto_id"`      // ID del boleto al que se le cambia el asiento
    NuevoAsiento  string `json:"nuevo_asiento"`   // Código del nuevo asiento solicitado
    CambiarAsientoVueloRequest

```

Representa la solicitud para cambiar el asiento asignado a un boleto dentro de una reservación de vuelo.

```

type CancelarReservacionRequest struct {
    Motivo string `json:"motivo"` // Motivo de la cancelación descrito por el usuario
    CancelarReservacionRequest

```

Representa la solicitud para cancelar una reservación, incluyendo el motivo de la cancelación proporcionado por el usuario.

```

type CrearProveedorRequest struct {
    Nombre          string    `json:"nombre" binding:"required"`    // Nombre comercial del proveedor
    TipoProveedorID int       `json:"tipo_proveedor_id" binding:"required"` // ID del tipo de proveedor (aerolínea u hotel)
    URLAPI          string    `json:"url_api"`                      // URL base de la API del proveedor
    UsuarioID       int       `json:"usuario_id" binding:"required"` // ID del usuario vinculado; debe tener rol 3
    PorcentajeGanancia float64 `json:"porcentaje_ganancia"`          // Porcentaje de ganancia aplicado sobre el precio base
    CrearProveedorRequest

```

Representa los datos necesarios para registrar un nuevo proveedor en el sistema de la agencia. El campo UsuarioID debe corresponder a un usuario con rol 3 (administrador de proveedor).

```

type CrearProveedorResponse struct {
    ID            int    `json:"id"`           // ID generado para el nuevo proveedor
    Nombre        string `json:"nombre"`       // Nombre comercial del proveedor
    TipoProveedorID int   `json:"tipo_proveedor_id"` // ID del tipo de proveedor registrado
    URLAPI        string `json:"url_api"`      // URL base de la API del proveedor
    UsuarioID     int   `json:"usuario_id"`    // ID del usuario vinculado al proveedor
    EstadoID      int   `json:"estado_id"`     // ID del estado inicial del proveedor
    PorcentajeGanancia float64 `json:"porcentaje_ganancia"` // Porcentaje de ganancia configurado
}
CrearProveedorResponse

```

Representa los datos del proveedor recién creado que se retornan al cliente tras el registro exitoso en el sistema.

```

type CrearReservacionRequest struct {
    TipoReservaID int `json:"tipo_reserva_id"` // ID del tipo de reserva (1=Aerolinea, 2=Hotel, 3=Paquete)
}
CrearReservacionRequest

```

Representa la solicitud para crear una nueva reservación, indicando el tipo de reserva que el usuario desea iniciar.

```

type CrearReservacionResponse struct {
    ID            int    `json:"id"`           // ID único generado para la reservación
    NoReservacion string `json:"no_reservacion"` // Número de reservación legible para el usuario
    EstadoID      int    `json:"estado_id"`    // ID del estado inicial de la reservación
    Estado        string `json:"estado"`       // Nombre del estado inicial de la reservación
    TipoReservaID int    `json:"tipo_reserva_id"` // ID del tipo de reserva creado
    FechaExpiracion string `json:"fecha_expiracion"` // Fecha y hora de expiración de la reservación
    FechaCreacion string `json:"fecha_creacion"`  // Fecha y hora de creación de la reservación
}
CrearReservacionResponse

```

Representa la respuesta retornada al cliente tras crear exitosamente una nueva reservación en el sistema.

```

type DetalleCompletoResponse struct {
    ID            int    `json:"id"`           // ID único del detalle
    TipoDetalleID int    `json:"tipo_detalle_id"` // Tipo de detalle: 1=Vuelo, 2=Hotel
    IDReservaProveedor string `json:"id_reserva_proveedor"` // ID de la reserva en el sistema del proveedor
    Total         float64 `json:"total"`        // Monto total del detalle
    EstadoDetalleID int    `json:"estado_detalle_id"` // ID del estado del detalle
    ParametrosJson interface{} `json:"parametros_json"` // Parametros adicionales almacenados como JSON
    DataProveedor interface{} `json:"data_proveedor"`  // Respuesta cruda retornada por el proveedor externo
}
DetalleCompletoResponse

```

Representa un detalle de reservación enriquecido con los datos locales de la agencia más la respuesta cruda obtenida del proveedor externo.

```

type DetalleProveedor struct {
    IDReservaProveedor string // ID de la reserva en el sistema del proveedor externo
    ProveedorID        int    // ID del proveedor en la base de datos de la agencia
    URLAPI              string // URL base de la API del proveedor
    TokenEntrada        string // Token de autenticacion para acceder a la API del proveedor
    TipoDetalleID       int    // ID del tipo de detalle (1=Vuelo, 2=Hotel)
    DetalleProveedor

```

Representa la informacion de un detalle de reservacion vinculado a un proveedor externo, incluyendo los datos de acceso a su API.

```

type DetalleResumenResponse struct {
    DetalleResumenResponse

```

Representa los datos de un detalle de reservacion guardados localmente en la base de datos de la agencia, sin datos del proveedor.

```

type FilaReservacionDetalle struct {
    ReservacionID      int    // ID unico de la reservacion
    NoReservacion       string // Numero de reservacion
    TipoReservaID      int    // Tipo de reserva
    EstadoID           int    // Estado de la reservacion
    Total              float64 // Monto total de la reservacion
    FechaCreacion       string // Fecha de creacion de la reservacion
    FechaExpiracion     *string // Fecha de expiracion (puede ser nula)
    DetalleID          int    // ID del detalle de reservacion
    TipoDetalleID      int    // Tipo del detalle (1=Vuelo, 2=Hotel)
    IDReservaProveedor string // ID de la reserva en el proveedor
    DetalleTotal        float64 // Monto total del detalle
    EstadoDetalleID    int    // Estado del detalle
    ParametrosJson      string // Parametros del detalle en formato JSON crudo
    ProveedorID         int    // ID del proveedor asociado al detalle
    URLAPI              string // URL base de la API del proveedor
    TokenEntrada        string // Token de autenticacion del proveedor
    FilaReservacionDetalle

```

Representa una fila plana del resultado de una consulta SQL que une datos de reservacion con sus detalles y la informacion del proveedor, utilizada para construir las respuestas estructuradas.

```

type HandshakeResponse struct {
    Mensaje      string `json:"mensaje"` // Mensaje descriptivo del resultado del handshake
    TokenSalida  string `json:"token_salida"` // Token de autenticacion devuelto por la aerolinea para sesiones posteriores
    HandshakeResponse

```

Representa la respuesta recibida del proveedor externo durante el proceso de handshake de autenticacion, que incluye el token de sesion devuelto por la aerolinea.

```

type HotelProveedorDTO struct {
    ID      int    `json:"id"`      // ID unico del hotel en el sistema del proveedor
    Nombre  string  `json:"nombre"`  // Nombre comercial del hotel
    Ciudad  string  `json:"ciudad"`  // Ciudad donde se encuentra el hotel
    Pais    string  `json:"pais"`    // Pais donde se encuentra el hotel
    HotelProveedorDTO

```

Representa la informacion basica de un hotel registrado en el catalogo de un proveedor hotelero externo.

```

type LoginRequest struct {
    Login      string `json:"login"`      // Correo electronico o nombre de usuario
    Contraseña string `json:"contrasena"` // Contraseña del usuario
    LoginRequest

```

Representa las credenciales enviadas por el usuario para iniciar sesion en el sistema de la agencia.

```

type LoginResponse struct {
    ID      int    `json:"id"`      // ID unico del usuario en la base de datos
    Nombre  string `json:"nombre"`  // Nombre del usuario
    Apellido string `json:"apellido"` // Apellido del usuario
    Correo  string `json:"correo"`   // Correo electronico del usuario
    Username string `json:"username"` // Nombre de usuario unico
    RolID   int    `json:"rol_id"`  // ID del rol asignado al usuario
    LoginResponse

```

Representa los datos del usuario autenticado que se retornan al cliente tras un inicio de sesion exitoso.

```

type ObtenerAsientosVueloRequest struct {
    ReservacionID int `json:"reservacion_id"` // ID de la reservacion en la base de datos de la agencia
    ProveedorID   int `json:"proveedor_id"`  // ID del proveedor aerolinea
    ObtenerAsientosVueloRequest

```

Representa la solicitud para consultar los asientos disponibles de un vuelo asociado a una reservacion especifica.

```

type PagoProveedorBody struct {
    Nit      string `json:"nit"`      // Numero de identificacion tributaria del cliente
    CodigoPostal string `json:"codigoPostal"` // Codigo postal de facturacion
    PagoProveedorBody

```

Representa el cuerpo de la solicitud que se envia al proveedor externo para procesar el pago de una reservacion en su sistema.

```

type PagoReservacionRequest struct {
    ReservacionID int    `json:"reservacion_id" binding:"required"` // ID de la reservacion a pagar
    TarjetaNumero string `json:"tarjeta_numero" binding:"required"` // Numero completo de la tarjeta de credito o
debito
    TarjetaCVV     string `json:"tarjeta_cvv" binding:"required"` // Codigo de verificacion de la tarjeta
    TarjetaMes     string `json:"tarjeta_mes" binding:"required"` // Mes de vencimiento de la tarjeta (formato
MM)
    TarjetaAnio    string `json:"tarjeta_anio" binding:"required"` // Anio de vencimiento de la tarjeta (formato
AAAA)
    Nit            string `json:"nit" binding:"required"` // Numero de identificacion tributaria del
cliente
    CodigoPostal   string `json:"codigo_postal" binding:"required"` // Codigo postal de facturacion
    PagoReservacionRequest

```

Representa la solicitud de pago enviada por el cliente para completar una reservacion, incluyendo los datos de la tarjeta y la informacion de facturacion.

```

type PasajeroVueloDTO struct {
    BoletoID int    `json:"boletoId"` // ID del boleto devuelto por la aerolinea
    Nombre   string `json:"nombre"` // Nombre del pasajero
    Apellido string `json:"apellido"` // Apellido del pasajero
    Pasaporte string `json:"pasaporte"` // Numero de pasaporte (solo digitos)
    Telefono string `json:"telefono"` // Numero de telefono de contacto
    Pais     string `json:"pais"` // Pais de residencia del pasajero
    Ciudad   string `json:"ciudad"` // Ciudad de residencia del pasajero
    PasajeroVueloDTO

```

Representa los datos personales de un pasajero asociado a un boleto especifico dentro de una reservacion de vuelo.

```

type ProveedorCatalogo struct {
    ProveedorID int // ID unico del proveedor en la base de datos de la agencia
    Nombre      string // Nombre comercial del proveedor
    URLApi      string // URL base de la API del proveedor
    TokenEntrada string // Token de autenticacion para acceder a la API del proveedor
    PorcentajeGanancia float64 // Porcentaje de ganancia aplicado al precio base del proveedor
    ProveedorCatalogo

```

Representa los datos de un proveedor obtenidos del catalogo interno de la agencia, utilizados para realizar consultas a su API externa.

```

type ProveedorIdentidad struct {
    ID          int    `json:"id"`           // ID unico del proveedor en la base de datos de la
    agencia
    Nombre      string `json:"nombre"`       // Nombre comercial del proveedor
    TipoProveedorID int  `json:"tipo_proveedor_id"` // ID del tipo de proveedor (aerolinea u hotel)
    ProveedorIdentidad

```

Representa los datos de identificacion minimos de un proveedor, utilizados para reconocer y clasificar al proveedor en operaciones internas sin cargar todos sus datos completos.

```

type ProveedorPuedeCancelarResponse struct {
    PuedeCancelar bool `json:"puedeCancelar"` // Indica si el proveedor permite la cancelacion
    Razon          string `json:"razon"`         // Mensaje explicativo del proveedor
    ProveedorPuedeCancelarResponse

```

Representa la respuesta que entrega el proveedor externo al consultar si una reserva puede ser cancelada en su sistema.

```

type RegistroUsuarioRequest struct {
    Nombre      string `json:"nombre"`       // Nombre del usuario
    Apellido    string `json:"apellido"`     // Apellido del usuario
    Correo      string `json:"correo"`       // Correo electronico del usuario
    Username    string `json:"username"`     // Nombre de usuario unico
    Contraseña  string `json:"contrasena"`   // Contraseña de acceso al sistema
    Pasaporte   string `json:"pasaporte"`    // Numero de pasaporte del usuario
    Telefono    string `json:"telefono"`     // Numero de telefono de contacto
    FechaNacimiento string `json:"fecha_nacimiento"` // Fecha de nacimiento en formato ISO 8601
    Ciudad      string `json:"ciudad"`       // Ciudad de residencia del usuario
    Pais        string `json:"pais"`         // Pais de residencia del usuario
    Nacionalidades []string `json:"nacionalidades"` // Lista de nacionalidades del usuario
    RegistroUsuarioRequest

```

Representa los datos enviados por un nuevo usuario para crear una cuenta en el sistema de la agencia de viajes.

```

type ReservacionConDetalles struct {
    ID          int // ID unico de la reservacion
    Detalles []DetalleProveedor // Lista de detalles asociados a proveedores externos
    ReservacionConDetalles

```

Representa una reservacion junto con la lista de sus detalles de proveedores, utilizado internamente para operaciones de negocio.

```

type ReservacionDetalladaResponse struct {
    ID            int           `json:"id"`           // ID unico de la reservacion
    NoReservacion string       `json:"no_reservacion"` // Numero de reservacion legible para el
usuario
    TipoReserva   int           `json:"tipo_reserva"` // Tipo de reserva: 1=Aerolinea, 2=Hotel,
3=Paquete
    EstadoID      int           `json:"estado_id"`    // ID del estado actual de la reservacion
    Total         float64      `json:"total"`        // Monto total de la reservacion
    FechaCreacion string       `json:"fecha_creacion"` // Fecha y hora de creacion de la
reservacion
    FechaExpiracion *string      `json:"fecha_expiracion"` // Fecha y hora de expiracion (puede ser
nula)
    Detalles       []DetalleCompletoResponse `json:"detalles"`      // Lista de detalles con datos locales y
del proveedor
    ReservacionDetalladaResponse

Representa la informacion completa de una reservacion combinando los datos
locales de la agencia con las respuestas de los proveedores externos.

```

```

type ReservacionResumenResponse struct {
    ID            int           `json:"id"`           // ID unico de la reservacion
    NoReservacion string       `json:"no_reservacion"` // Numero de reservacion legible para el
usuario
    TipoReserva   int           `json:"tipo_reserva"` // Tipo de reserva: 1=Aerolinea, 2=Hotel,
3=Paquete
    EstadoID      int           `json:"estado_id"`    // ID del estado actual de la reservacion
    Total         float64      `json:"total"`        // Monto total de la reservacion
    FechaCreacion string       `json:"fecha_creacion"` // Fecha y hora de creacion de la
reservacion
    FechaExpiracion *string      `json:"fecha_expiracion"` // Fecha y hora de expiracion (puede ser
nula)
    Detalles       []DetalleResumenResponse `json:"detalles"`      // Lista de detalles asociados a la
reservacion
    ReservacionResumenResponse

Representa el resumen de una reservacion con sus datos locales, utilizado
para listar las reservaciones de un usuario sin consultar informacion
adicional a los proveedores externos.

```

```

type ReservacionValidada struct {
    ID            int // ID unico de la reservacion
    EstadoID      int // ID del estado actual de la reservacion
    TipoReservaID int // ID del tipo de reservacion (1=Aerolinea, 2=Hotel, 3=Paquete)
    UsuarioID     int // ID del usuario propietario de la reservacion
    ReservacionValidada

Representa los datos minimos de una reservacion ya verificada que se usan
internamente para validar operaciones posteriores.

```

```

type RutaProveedorDTO struct {
    ID          int    `json:"id"`           // ID unico de la ruta en el sistema del proveedor
    CiudadOrigen string `json:"ciudadOrigen"` // Ciudad de origen del vuelo
    PaisOrigen  string `json:"paisOrigen"`   // Pais de origen del vuelo
    CiudadDestino string `json:"ciudadDestino"` // Ciudad de destino del vuelo
    PaisDestino string `json:"paisDestino"`   // Pais de destino del vuelo
    Duracion    int    `json:"duracion"`     // Duracion del vuelo en minutos
}
RutaProveedorDTO

```

Representa la informacion de una ruta de vuelo devuelta por el proveedor aerolinea, incluyendo origen, destino y duracion.

```

type SeleccionHabitacion struct {
    HabitacionID    int    `json:"habitacionId"` // ID de la habitacion en el catalogo del proveedor
    CantidadPersonas int    `json:"cantidadPersonas"` // Numero de personas que ocuparan la habitacion
}
SeleccionHabitacion

```

Representa una habitacion especifica elegida por el usuario, con las fechas de estancia y la cantidad de personas.

```

type SeleccionVuelo struct {
    VueloID        int    `json:"vueloId"`           // ID del vuelo en el catalogo del proveedor
    ClaseID        int    `json:"claseId"`           // ID de la clase de servicio seleccionada
    CantidadPasajeros int    `json:"cantidadPasajeros"` // Numero de pasajeros para este vuelo
}
SeleccionVuelo

```

Representa un vuelo especifico elegido por el usuario, junto con la clase de servicio y la cantidad de pasajeros.

```

type UsuarioResumen struct {
    RolID    int    `json:"rolId"` // ID del rol asignado al usuario
    Rol      string `json:"rol"`   // Nombre del rol asignado al usuario
}
UsuarioResumen

```

Representa los datos basicos de un usuario retornados por el panel de administracion para gestion de roles y asignacion de WebService.

```

type ValidacionUsuarioResponse struct {
    Correo    bool `json:"correo"` // Indica si el correo ya esta registrado en el sistema
    Pasaporte bool `json:"pasaporte"` // Indica si el pasaporte ya esta registrado en el sistema
    Username  bool `json:"username"` // Indica si el nombre de usuario ya esta registrado en el sistema
}
ValidacionUsuarioResponse

```

Representa el resultado de la validacion de unicidad de los datos de un usuario durante el proceso de registro, indicando si el correo, pasaporte o username ya existen en el sistema.

```
type VerificarCancelacionResponse struct {
    PuedeCancelar bool           `json:"puede_cancelar"` // Indica si la reservacion completa puede
cancelarse
    Detalles      []VerificarDetalleResponse `json:"detalles"`      // Lista de verificaciones por detalle de
reservacion
    VerificarCancelacionResponse

    Representa la respuesta de verificacion previa a la cancelacion, indicando
    si la reservacion puede cancelarse y el detalle por cada componente.
```

```
type VerificarDetalleResponse struct {
    TipoDetalleID    int    `json:"tipo_detalle_id"` // ID del tipo de detalle (1=Vuelo, 2=Hotel)
    IDReservaProveedor string `json:"id_reserva_proveedor"` // ID de la reserva en el sistema del proveedor
    PuedeCancelar    bool   `json:"puede_cancelar"` // Indica si este detalle puede cancelarse
    Razon            string `json:"razon"`          // Razon del resultado de la verificacion
    VerificarDetalleResponse

    Representa el resultado de verificacion de cancelacion para un detalle
    especifico de la reservacion (vuelo u hotel).
```

Helpers

Provee funciones auxiliares reutilizables para tareas comunes de la aplicacion

Movent: generacion de tokens, hashing de contraseñas, manejo de sesiones JWT, envio de correos electronicos y generacion de documentos PDF.

FUNCTIONS

```
func BuildHTMLBienvenida(nombre, apellido, username, correo, telefono, fechaNacimiento, ciudad, pais string, nacionalidades []string) string
```

```
BuildHTMLBienvenida
```

Genera el HTML completo del correo de bienvenida que se envia al usuario tras completar el registro. Incluye una tabla con los datos de la cuenta recién creada.

Parametros:

- nombre: nombre del usuario registrado
- apellido: apellido del usuario registrado
- username: nombre de usuario elegido
- correo: direccion de correo electronico registrada
- telefono: numero de telefono del usuario
- fechaNacimiento: fecha de nacimiento en formato string
- ciudad: nombre de la ciudad del usuario
- pais: nombre del pais del usuario
- nacionalidades: slice con los nombres de las nacionalidades del usuario

Retorna:

- string: documento HTML completo listo para enviar como cuerpo de correo

```
func BuildHTMLEmail(data ReservacionPDFData) string
```

```
BuildHTMLEmail
```

Genera el HTML completo del correo de confirmacion de reservacion. Incluye tarjetas para boletos de vuelo y habitaciones de hotel con todos los detalles de la reservacion en formato de tabla.

- data: struct ReservacionPDFData con todos los datos de la reservacion

```
func CheckPassword(password, hash string) bool
```

```
CheckPassword
```

Compara una contraseña en texto plano contra su hash bcrypt almacenado.
Retorna verdadero si coinciden, falso en cualquier otro caso.

- password: contraseña en texto plano ingresada por el usuario
- hash: hash bcrypt almacenado en la base de datos
- bool: true si la contraseña es valida, false si no coincide

```
func EnviarBienvenida(correo, nombre, apellido, username, telefono, fechaNacimiento, ciudad, pais string, nacionalidades []string) error
```

```
EnviarBienvenida
```

Construye el correo HTML de bienvenida con los datos del usuario recién registrado y lo envía a su dirección de correo electrónico.

- correo: dirección de correo del nuevo usuario
- nombre: nombre del usuario
- apellido: apellido del usuario
- error: error si la construcción del HTML o el envío SMTP falla

```
func EnviarEmailConPDF(destinatario, asunto, htmlBody string, pdfBytes []byte, nombreArchivo string) error
```

```
EnviarEmailConPDF
```

Construye y envía un correo electrónico multipart con cuerpo HTML y un archivo PDF adjunto. Ambas partes se codifican en base64. La conexión SMTP se establece con STARTTLS y autenticación PLAIN.

- destinatario: dirección de correo del receptor
- asunto: línea de asunto del mensaje
- htmlBody: contenido HTML del cuerpo del correo
- pdfBytes: bytes del archivo PDF a adjuntar
- nombreArchivo: nombre con el que se adjunta el PDF
- error: error si la configuración SMTP es incompleta o el envío falla

```
func EnviarEmailHTML(destinatario, asunto, htmlBody string) error
```

```
EnviarEmailHTML
```

Construye y envía un correo electrónico con cuerpo HTML puro, sin archivos adjuntos. El cuerpo se codifica en base64 antes del envío. Se usa para notificaciones simples como la bienvenida.

func GenerarPDFReservacion(data ReservacionPDFData) ([]byte, error)

GenerarPDFReservacion

Genera el documento PDF de comprobante de reservacion a partir de los datos proporcionados. Construye el layout en dos columnas con un header fijo, banda de resumen, datos generales, boletos o habitaciones, total y condiciones. Retorna los bytes del PDF.

- data: struct ReservacionPDFData con toda la informacion de la reservacion
- []byte: contenido binario del PDF generado
- error: error si la generacion o escritura del PDF falla

func GenerarToken(usuarioID int, username string, rolID int) (string, error)

GenerarToken

Crea y firma un JWT HS256 con los datos del usuario autenticado. El token tiene una vigencia de 24 horas a partir del momento de su emision y se firma con la clave JWT_SECRET del entorno.

- usuarioID: identificador unico del usuario en la base de datos
- username: nombre de usuario para incluir en los claims
- rolID: identificador del rol del usuario para control de acceso
- string: token JWT firmado listo para enviar al cliente
- error: error si la firma del token falla

func GenerarTokenHash() (string, error)

GenerarTokenHash

Genera un token aleatorio seguro de 32 bytes y retorna su hash SHA-256 codificado en hexadecimal. Se usa para crear tokens de autentificacion o identificacion de proveedores.

- string: cadena hexadecimal de 64 caracteres con el hash SHA-256
- error: error si la lectura de bytes aleatorios falla

func HashPassword(password string) (string, error)

HashPassword

Genera el hash bcrypt de la contraseña en texto plano proporcionada, usando el costo por defecto de la librería bcrypt.

- password: contraseña en texto plano a hashear
- string: hash bcrypt resultante
- error: error si la generacion del hash falla

TYPES

```
type BoletoPDF struct {  
    NoBoleto      string  
    NumeroVuelo   string  
    Clase         string  
    NoAsiento     string  
    OrigenCodigo  string  
    OrigenCiudad  string  
    DestinoCodigo string  
    DestinoCiudad string  
    HoraSalida    string  
    HoraLlegada   string  
    FechaVuelo    string  
    AvionMarca    string  
    AvionModelo   string  
    Precio        float64  
    EstadoBoleto  string  
    PasajeroNombre string  
}
```

BoletoPDF

Representa los datos de un boleto de vuelo para incluir en el PDF o correo de confirmacion de reservacion.

```
type Claims struct {  
    UsuarioID int    `json:"usuario_id"`  
    Username  string `json:"username"`  
    RolID     int    `json:"rol_id"`  
    jwt.RegisteredClaims  
    Claims
```

Estructura de los claims personalizados incluidos en el JWT de sesion.
Extiende jwt.RegisteredClaims con los datos de identidad del usuario necesarios para autorizacion en los middlewares y controladores.

```
func VerificarToken(tokenStr string) (*Claims, error)
```

VerificarToken

Parsea y valida un JWT firmado con HS256. Verifica que el metodo de firma sea HMAC, que la firma sea correcta y que el token no haya expirado, retornando los claims si todo es valido.

- tokenStr: cadena JWT recibida desde el cliente
- *Claims: puntero a los claims extraidos del token valido
- error: error si el metodo de firma es invalido, la firma no coincide, el token esta expirado o los claims no son validos

```
type HabitacionPDF struct {
    NombreHotel      string
    TipoHabitacion   string
    TipoCama         string
    NumeroHabitacion string
    FechaCheckIn     string
    FechaCheckOut    string
    CantidadPersonas int
    TotalDetalle     float64
    Estado           string
}
HabitacionPDF
```

Representa los datos de una habitacion de hotel para incluir en el PDF o correo de confirmacion de reservacion.

```
type ReservacionPDFData struct {
    NoReservacion string
    EstadoReserva string
    FechaCreacion string
    Total          float64
    TipoReserva    int
    UsuarioNombre  string
    UsuarioEmail   string
    Boletos        []BoletoPDF
    Habitaciones   []HabitacionPDF
}
ReservacionPDFData
```

Agrupar todos los datos necesarios para generar el PDF y el correo HTML de una reservacion. Contiene informacion del encabezado, del usuario titular y las listas de boletos y habitaciones.

```
type SMTPConfig struct {
    Host      string
    Port      string
    User      string
    Password  string
    From      string
}
SMTPConfig
```

Contiene los parametros de conexion al servidor de correo SMTP. Se pobla desde variables de entorno mediante GetSMTPConfig.

```
func GetSMTPConfig() SMTPConfig
GetSMTPConfig
```

Lee las variables de entorno SMTP y retorna un SMTPConfig listo para usar. Si alguna variable no esta definida se usa el valor por defecto indicado (host: smtp.gmail.com, puerto: 587).

- SMTPConfig: struct con los datos de conexion SMTP

INFRAESTRUCTURA

Middlewares

Contiene los middlewares HTTP de la aplicacion Movent. Provee funciones de verificacion de autenticacion JWT, control de roles y validacion de tokens de proveedor para proteger las rutas de la API REST.

FUNCTIONS

func AuthRequerido() gin.HandlerFunc

AuthRequerido

Retorna un middleware de Gin que verifica que la solicitud incluya una cookie de sesion con un JWT valido. Si la cookie no existe o el token es invalido o esta expirado, la solicitud es rechazada con HTTP 401. Si la autenticacion es exitosa, inyecta los datos del usuario (usuario_id, username, rol_id) en el contexto de Gin para que los controladores puedan usarlos.

Retorna:

- gin.HandlerFunc: funcion de middleware lista para usar con router.Use

func ProveedorRequerido(db *sql.DB) gin.HandlerFunc

ProveedorRequerido

Retorna un middleware de Gin que valida el token de proveedor enviado en el encabezado HTTP X-Proveedor-Token. Busca el proveedor correspondiente en la base de datos mediante el repositorio. Si el token esta ausente, no se encuentra o produce un error, rechaza la solicitud con HTTP 401 o 500 segun corresponda. Si es valido, inyecta proveedor_id, proveedor_nombre y proveedor_tipo en el contexto de Gin.

Parametros:

- db: conexion activa a la base de datos para consultar el proveedor

```
func RolRequerido(roles ...int) gin.HandlerFunc
```

RolRequerido

Retorna un middleware de Gin que verifica que el usuario autenticado tenga uno de los roles permitidos indicados como argumentos. Lee el rol del contexto de Gin (previamente inyectado por AuthRequerido) y compara con los roles aceptados. Si el rol no coincide con ninguno, rechaza la solicitud con HTTP 403.

- roles: uno o mas identificadores de rol que tienen acceso permitido

Notas:

- Debe usarse siempre despues de AuthRequerido en la cadena de middlewares

Config

Gestiona la configuracion de la aplicacion Movent. Lee las variables de entorno necesarias para la conexion a la base de datos y el servidor HTTP, con soporte para archivo .env mediante la libreria godotenv.

TYPES

```
type Config struct {
    DBHost      string
    DBPort      string
    DBUser      string
    DBPassword  string
    DBName      string
    ServerPort  string
    ServerURL   string
}
```

Config

Contiene todos los parametros de configuracion necesarios para iniciar la aplicacion: credenciales de la base de datos MySQL, puerto del servidor HTTP y URL base del servidor.

```
func Load() *Config
```

Load

Carga la configuracion desde variables de entorno. Intenta leer un archivo .env en el directorio de trabajo; si no existe, utiliza directamente las variables del sistema operativo.

Retorna:

- *Config: puntero al struct poblado con los valores del entorno

Notas:

- Si el archivo .env no se encuentra solo se registra un aviso en el log; la funcion no falla por esa causa

Database

Provee la conexión a la base de datos MySQL de Movent. Expone una única función `Connect` que construye el DSN a partir de la configuración de la aplicación y retorna una instancia verificada de `*sql.DB` lista para ser usada por los repositorios.

FUNCTIONS

func Connect(cfg *config.Config) *sql.DB

Connect

Establece y verifica la conexión a MySQL usando la configuración provista. Construye el DSN con los campos del struct `Config`, abre la conexión y ejecuta un Ping para confirmar disponibilidad antes de retornar.

Parametros:

- `cfg`: puntero a la configuración con credenciales y host de la BD

Retorna:

- `*sql.DB`: instancia de la conexión lista para usar

Notas:

- Termina el proceso con `log.Fatal` si la apertura de la conexión falla
- Termina el proceso con `log.Fatal` si el Ping a la base de datos falla